Tejiendo redes: arquitectura de software entre hilos y nodos

NexusAI 2.0: de la arquitectura a la red

Juan Carlos Benítez Rojas

Líder de proyecto y arquitecto de software

Juan José Córdoba Eraso

Desarrollador y evaluador de calidad

Corporación Escuela Tecnológica del Oriente

Ingeniería de Software

Arquitectura de Software — Unidad 2

Leidy Johana Polo

11 de noviembre de 2026

Resumen

Este artículo documenta el diseño, la implementación y la evaluación de NexusAI 2.0, un prototipo distribuido que materializa la arquitectura planteada en la Actividad 2 para una plataforma de atención automatizada. El sistema se compone de un API Gateway que orquesta Sagas, un broker de eventos con bitácora inmutable y tres nodos de cómputo con pools de hilos, comunicados exclusivamente por HTTP y mensajes. Se aplicaron nueve patrones —Saga orquestada, Broker, Mediator, Proxy, Circuit Breaker, Bulkhead, Observer distribuido, Strategy y Retry— y se evaluó el comportamiento con una suite de nueve escenarios y 1 200 transacciones. El escalado horizontal fue casi lineal: 9,24, 18,14 y 26,74 transacciones por segundo con uno, dos y tres nodos, un speedup de 2,89× y una eficiencia del 96,5 %. El sistema atendió el 100 % de las transacciones en todos los escenarios de fallo, incluidos un nodo defectuoso, un nodo apagado y un 25 % de rechazos compensados por la Saga. Se documentan cinco ajustes derivados de la medición, entre ellos el reemplazo de `localhost` por `127.0.0.1`, que redujo la latencia por petición de 2 042 ms a 2,15 ms, y la sustitución de round-robin por least-connections, que aumentó el throughput bajo un nodo lento de 8,99 a 20,59 transacciones por segundo.

Palabras clave: arquitectura de software, sistemas distribuidos, patrones de diseño, Saga, Circuit Breaker, escalabilidad, tolerancia a fallos, Python

Índice

Introducción 6

Contexto del proyecto 6

Propósito del diseño arquitectónico 7

Objetivos 7

Descripción del sistema distribuido 8

Tipo de sistema y dominio 8

Componentes funcionales 9

Supuestos y restricciones 10

Herramientas seleccionadas 11

Análisis y planificación del proyecto 12

Requisitos funcionales 12

Requisitos no funcionales 13

Relevancia de la comunicación entre nodos e hilos 13

Metas del sistema y métricas de éxito 15

Estrategia de trabajo, roles y cronograma 15

Modelado UML 17

Diagrama de componentes 18

Diagrama de secuencia 20

Diagrama de despliegue 22

Selección y justificación de patrones de diseño 23

Saga orquestada 23

Broker de mensajes 24

Mediator 24

Proxy y API Gateway 25

Circuit Breaker 25

Bulkhead 26

Observer distribuido 26

Strategy 27

Retry 27

Síntesis de patrones 28

Implementación y simulación 28

Estructura del código 29

Modelo de procesos, hilos y sincronización 30

Contrato HTTP y flujo de una transacción 31

Simulación de las etapas cognitivas 32

Puesta en marcha y reproducibilidad 33

Observabilidad y trazabilidad 34

Evaluación y ajustes 35

Metodología experimental 36

Escalabilidad horizontal 37

Robustez ante fallos 39

Costo por paso y sobrecarga de coordinación 42

Ajustes derivados de la medición 43

Cumplimiento de las metas 46

Limitaciones y validez 47

Conclusiones 48

Referencias 53

Anexo A. Contratos HTTP del prototipo 55

Anexo B. Extracto de la bitácora de eventos 57

Anexo C. Salida de la suite de evaluación 58

Anexo D. Reproducción del entorno 60

Anexo E. Enlaces de entrega 61

Introducción

Contexto del proyecto

La presente actividad se desarrolla en el marco de la Unidad 2 del curso de Arquitectura de Software, denominada Patrones y diseño de arquitectura, y responde al mini proyecto integrador Tejiendo redes: Arquitectura de software entre hilos y nodos. El propósito formativo es que el equipo demuestre su capacidad para diseñar, modelar, implementar y evaluar una arquitectura distribuida, aplicando técnicas de especificación UML y seleccionando patrones de diseño pertinentes para entornos de computación en la nube.

El proyecto retoma el dominio construido en la Actividad 2, la plataforma NexusAI, concebida como una arquitectura cloud-native para la orquestación autónoma de agentes de inteligencia artificial empresarial. Aquel ejercicio produjo un diseño conceptual sólido, con patrones como Saga, Circuit Breaker, CQRS, Bulkhead y API Gateway, pero dejó tres brechas que la evaluación de la rúbrica actual hace visibles: no existía un diagrama de componentes, la concurrencia entre nodos e hilos no estaba especificada y el prototipo era un tablero estático que simulaba latencias con temporizadores del navegador. En consecuencia, no había forma de medir escalabilidad ni robustez, solo de declararlas.

La Actividad 4 cierra esas brechas con una decisión explícita: NexusAI evoluciona de una arquitectura dibujada a una red ejecutable. El equipo conserva el dominio, los patrones y la narrativa del proyecto anterior, y añade un prototipo distribuido real compuesto por procesos independientes que se comunican por HTTP y por un bus de eventos, con hilos de ejecución verificables, pruebas de carga concurrente y escenarios de fallo controlado. La docente autorizó expresamente reutilizar la temática del proyecto anterior, de modo que esta continuidad no es un atajo, sino una decisión de diseño coherente con la evaluación por competencias.

Propósito del diseño arquitectónico

El propósito del diseño es demostrar que una arquitectura distribuida bien especificada puede sostener tres propiedades medibles: escalabilidad horizontal, entendida como la capacidad de aumentar el throughput al incorporar nodos de cómputo; robustez, entendida como la capacidad de mantener la atención del servicio cuando un nodo degrada su desempeño o falla por completo; y mantenibilidad, entendida como la posibilidad de razonar sobre el sistema a partir de contratos explícitos, responsabilidades separadas y patrones reconocibles.

Estas tres propiedades se traducen en decisiones concretas. La escalabilidad se aborda con nodos sin estado compartido y un balanceador que distribuye pasos de trabajo. La robustez se aborda con aislamiento de fallos mediante Circuit Breaker y Bulkhead, y con consistencia eventual mediante una Saga orquestada que compensa los pasos ya ejecutados. La mantenibilidad se aborda con un contrato HTTP/JSON uniforme, un broker que desacopla la publicación de eventos y un tablero de observabilidad que expone el estado interno del sistema en tiempo real.

Objetivos

Objetivo general

Diseñar, modelar, implementar y evaluar un prototipo de arquitectura distribuida para la plataforma NexusAI, aplicando patrones de diseño que garanticen comunicación eficiente entre nodos e hilos, escalabilidad horizontal y robustez ante fallos, con evidencia experimental de su desempeño.

Objetivos específicos

- Modelar la arquitectura distribuida con un diagrama de componentes, un diagrama de secuencia que evidencie la interacción entre nodos e hilos, y un diagrama de despliegue que describa la distribución física y lógica del sistema en la red.
- Seleccionar y justificar patrones de diseño adecuados al contexto distribuido, con base en la bibliografía del curso y en decisiones de arquitectura trazables.
- Implementar un prototipo funcional con procesos independientes, comunicación por red, hilos de ejecución concurrentes y un bus de eventos para la trazabilidad.
- Evaluar la escalabilidad del prototipo mediante pruebas de carga con uno, dos y tres nodos de cómputo, y la robustez mediante la inyección de latencia, fallos de nodo, fallos de negocio y caída total de un nodo.
- Documentar los resultados, identificar cuellos de botella y aplicar ajustes verificables que mejoren el rendimiento y la resiliencia del sistema.

Descripción del sistema distribuido

Tipo de sistema y dominio

NexusAI 2.0 es un sistema distribuido de procesamiento de tareas cognitivas orientado a servicios. Su función es recibir solicitudes omnicanal —por ejemplo, consultas provenientes de la web, de una aplicación móvil, de un canal de mensajería o de un centro de llamadas— y ejecutarlas a través de un pipeline de tres etapas: clasificación de la intención (triage), recuperación de contexto (rag) y validación de cumplimiento (compliance). Cada etapa representa, en una implementación productiva, la invocación de un modelo de lenguaje o de un servicio cognitivo especializado.

El sistema adopta una topología de orquestación centralizada con cómputo distribuido. Un nodo coordinador actúa como API Gateway, orquestador de la Saga y balanceador de carga; un nodo de mensajería opera como bus de eventos y bitácora de auditoría; y tres nodos trabajadores ejecutan los pasos del pipeline en pools de hilos independientes. Esta separación permite escalar el cómputo sin replicar la lógica de coordinación, aislar el fallo de un trabajador y auditar cada transición de estado.

La elección de una arquitectura orientada a servicios, en lugar de un monolito, se justifica porque el dominio presenta tres características que favorecen la distribución: etapas con costos heterogéneos, necesidad de degradación parcial y requisitos de auditoría. En un monolito, una etapa lenta bloquearía a las demás y un fallo en el cumplimiento invalidaría toda la solicitud sin dejar rastro compensable. En la arquitectura propuesta, cada etapa se ejecuta en un nodo que puede aislarse y cada transición se publica como un evento.

Componentes funcionales

El componente de presentación agrupa al cliente omnicanal, representado por el generador de carga concurrente, y al tablero operacional. El generador de carga emite transacciones completas contra el gateway y mide la latencia extremo a extremo; el tablero consume las métricas del coordinador y del broker para mostrar el estado de los nodos, la profundidad de las colas, la latencia por paso y las últimas transacciones.

El componente de coordinación reside en el nodo 1. El API Gateway expone el contrato HTTP/JSON y valida la estructura de la solicitud. El orquestador de la Saga mantiene la máquina de estados de cada transacción, decide el orden de los pasos y ejecuta la compensación inversa cuando una etapa falla. El balanceador selecciona el nodo de destino mediante una estrategia

intercambiable. El Circuit Breaker y la política de reintentos protegen al sistema de nodos degradados. El pool de despacho administra los hilos que aceptan transacciones concurrentes.

El componente de mensajería reside en el nodo 2. El broker mantiene topics en memoria, publica cada evento en un archivo JSONL inmutable y expone métricas de profundidad de cola y de eventos publicados. Los topics principales son events, para la traza de la Saga, y audit, para la persistencia. El broker desacopla a los productores de los consumidores y permite que el tablero y la bitácora evolucionen sin modificar a los nodos.

El componente de cómputo está formado por los nodos 3, 4 y 5. Cada trabajador es un proceso independiente que recibe un paso por HTTP, lo ejecuta dentro de su propio ThreadPool y devuelve el resultado. Los trabajadores no se conocen entre sí ni conocen al coordinador como cliente: solo responden a su contrato. Esta ignorancia mutua es la que permite agregar o retirar nodos sin reconfigurar el resto del sistema.

El componente de observabilidad atraviesa todos los nodos. Cada proceso expone un endpoint de salud y un endpoint de métricas. El coordinador agrega las métricas de las transacciones y del estado de los breakers; el broker agrega las métricas de los topics; los trabajadores reportan hilos activos, tareas completadas, errores y percentiles de latencia.

Supuestos y restricciones

El prototipo opera bajo supuestos explícitos. Se asume una red local confiable en cuanto a conectividad, pero no en cuanto a latencia ni disponibilidad: los escenarios de caos demuestran que el sistema debe tolerar nodos lentos y nodos ausentes. Se asume que las latencias de las etapas cognitivas son proporcionales a las de un modelo de lenguaje real, por lo que se simulan con valores de 80, 180 y 120 milisegundos, respectivamente, más una componente aleatoria. No

se invocan modelos de lenguaje reales porque ello introduciría costos variables, límites de cuota y una latencia no determinista que impediría comparar escenarios.

Se asume también que el estado de cada transacción cabe en memoria y que la duración máxima de una Saga es acotada. Esta simplificación es razonable para un prototipo académico, pero en producción exigiría un almacén durable de estados, como el que la Actividad 2 asignaba a Redis Enterprise. Finalmente, se asume que la compensación de un paso es idempotente: repetirla no produce efectos adversos, condición necesaria para que la Saga sea segura ante reintentos.

Las restricciones del proyecto son de tiempo, equipo y hardware. El equipo dispone de dos integrantes y de dos días efectivos de trabajo, y el entorno de pruebas es un equipo con cuatro núcleos lógicos y sistema operativo Windows 11. Por ello se privilegió la biblioteca estándar de Python, que elimina dependencias externas para el clúster y permite reproducir el experimento en cualquier equipo con Python 3.11 o superior. Docker no se utilizó en la ejecución, aunque el diagrama de despliegue documenta la red bridge equivalente, de modo que la migración a contenedores no requiera rediseño.

Herramientas seleccionadas

Tabla 1

Herramientas y tecnologías utilizadas en el proyecto

Herramienta	Versión	Uso en el proyecto	Justificación
Python	3.14.7	Implementación del clúster	Biblioteca estándar suficiente para sockets, hilos y HTTP
http.server / ThreadingHTTPServer	Estándar	Servidores HTTP de cada nodo	Un hilo por conexión, sin frameworks externos
concurrent.futures	Estándar	Pools de hilos de coordinador y workers	API de alto nivel para concurrencia y futuros
queue	Estándar	Colas thread-safe del broker y de eventos	Sincronización probada y simple

Herramienta	Versión	Uso en el proyecto	Justificación
urllib.request	Estándar	Cliente HTTP entre nodos	Sin dependencias, con timeouts explícitos
PlantUML	1.2026.9	Diagramas UML	Recomendado por la docente; versionado en texto
matplotlib / numpy	3.11 / 2.5	Gráficas de resultados	Series comparables y exportación PNG
Microsoft Word	Office 16	Informe APA 7 y exportación a PDF	Formato institucional solicitado
GitHub	—	Repositorio del prototipo	Entrega y trazabilidad de commits
YouTube (no listado)	—	Video de presentación	Requisito de la guía

Análisis y planificación del proyecto

Requisitos funcionales

Tabla 2

Requisitos funcionales del sistema distribuido

Código	Requisito	Criterio de aceptación
RF-01	Recibir solicitudes omnicanal por HTTP/JSON	POST /task responde 200, 409 o error controlado
RF-02	Ejecutar el pipeline triage, rag y compliance	Cada Saga registra los tres pasos con su nodo y latencia
RF-03	Orquestar la Saga con compensación inversa	Ante fallo, se compensan los pasos previos en orden inverso
RF-04	Distribuir carga entre nodos de cómputo	El balanceo reparte pasos entre los nodos habilitados
RF-05	Aislar nodos degradados o caídos	El Circuit Breaker deja de enrutar al nodo tras tres fallos
RF-06	Publicar eventos de cada transición	El broker persiste los eventos en un archivo JSONL
RF-07	Exponer métricas operacionales	/metrics reporta throughput, percentiles y estado de nodos
RF-08	Permitir pruebas de caos controladas	/chaos inyecta latencia y tasa de fallo por nodo

Requisitos no funcionales

Tabla 3

Requisitos no funcionales y su relación con ISO/IEC 25010

Código	Requisito	Categoría ISO/IEC 25010	Métrica objetivo
RNF-01	Escalar horizontalmente	Capacidad (performance)	Throughput crece al menos 2,5 veces con tres nodos
RNF-02	Sostener el servicio ante un nodo caído	Fiabilidad	100 % de transacciones atendidas con un nodo menos
RNF-03	Responder con latencia acotada	Eficiencia de desempeño	p95 menor a 1 500 ms con tres nodos
RNF-04	Recuperar el enrutamiento tras una caída	Recuperabilidad	El nodo reiniciado recibe carga en menos de 10 s
RNF-05	Garantizar consistencia eventual	Fiabilidad	Toda Saga termina completada o compensada, nunca a medias
RNF-06	Trazar cada transacción	Auditabilidad	Un evento por transición, con identificador de Saga
RNF-07	Operar sin dependencias externas	Portabilidad	Ejecutable con la biblioteca estándar de Python
RNF-08	Ser comprensible y modificable	Mantenibilidad	Un módulo por responsabilidad y contratos explícitos

Relevancia de la comunicación entre nodos e hilos

La comunicación entre nodos e hilos es el núcleo del ejercicio porque determina dónde aparece la latencia, dónde se forman las colas y cómo se propaga un fallo. En el prototipo conviven cuatro niveles de concurrencia. El primero es el nivel de proceso: cada nodo es un proceso del sistema operativo con su propio espacio de memoria, de modo que un fallo de memoria o una excepción no capturada no derriba a los demás. El segundo es el nivel de servidor: cada nodo usa un servidor HTTP multi-hilo, que atiende cada conexión en un hilo independiente y permite que un nodo reciba varias peticiones simultáneas sin bloquear el

aceptador. El tercero es el nivel de despacho: el coordinador mantiene un pool de dieciséis hilos que ejecutan Sagas completas, lo que acota la concurrencia admitida y protege al sistema de una sobrecarga descontrolada. El cuarto es el nivel de cómputo: cada trabajador mantiene un pool de cuatro hilos, que actúa como mamparo o bulkhead y limita cuántos pasos puede ejecutar en paralelo.

Esta jerarquía produce un efecto medible. Con tres nodos y cuatro hilos por nodo, la capacidad instantánea de cómputo es de doce pasos simultáneos. Si la concurrencia del cliente supera ese valor, las peticiones esperan en la cola del pool de despacho o en la cola interna del trabajador, y la latencia extremo a extremo crece aunque el throughput se mantenga estable. Esa diferencia entre latencia y throughput es precisamente la que se observa en los resultados: al pasar de uno a tres nodos, el throughput se multiplica por 2,88 mientras la latencia p95 se reduce a un tercio.

La comunicación se realiza mediante HTTP/JSON sobre TCP. Cada llamada entre el coordinador y un trabajador tiene un timeout explícito de diez segundos, y cada reintento se dirige a un nodo distinto del que falló. La publicación de eventos es asíncrona: el trabajador encola el evento y un hilo publicador lo envía al broker, de modo que una falla del broker no bloquea la respuesta al usuario. Esta decisión sacrifica garantía de entrega inmediata a cambio de disponibilidad, un intercambio clásico en sistemas distribuidos que se alinea con el teorema CAP y con la noción de consistencia eventual descrita por Kleppmann (2017).

La sincronización interna se resuelve con candados de grano fino. El broker protege sus colas y contadores con un lock; el coordinador protege su lista de trabajadores, los contadores de vuelo y las métricas con un lock; cada trabajador protege sus contadores y su lista de latencias con un lock propio. No hay memoria compartida entre nodos, lo que elimina la necesidad de

coordinación distribuida para el estado y convierte al sistema en un ejemplo de arquitectura shared-nothing.

Metas del sistema y métricas de éxito

El equipo definió metas verificables antes de implementar, con el fin de evitar una evaluación retrospectiva sesgada. La meta de escalabilidad exige que el throughput con tres nodos sea al menos dos veces y media el de un solo nodo, y que la eficiencia de escalado —el cociente entre el speedup y el número de nodos— se mantenga por encima del noventa por ciento. La meta de latencia exige un p95 inferior a mil quinientos milisegundos con tres nodos y una concurrencia de veinticuatro clientes. La meta de robustez exige que el cien por ciento de las transacciones se atienda cuando un nodo falla por completo, y que el cien por ciento de las transacciones con fallo de negocio termine compensada, sin errores de sistema. La meta de trazabilidad exige que cada transición produzca un evento y que el broker no pierda eventos por desbordamiento de cola.

Estas metas se eligieron porque cubren las dos dimensiones que la rúbrica evalúa con mayor peso: la implementación del prototipo y el uso de patrones. Un prototipo que solo funciona en el camino feliz no demuestra robustez; uno que solo resiste fallos pero no escala no demuestra arquitectura. La combinación de ambas pruebas es la que permite afirmar que el diseño es adecuado, y no solo que el código compila.

Estrategia de trabajo, roles y cronograma

El equipo aplicó una estrategia incremental con verificación temprana. Primero se modeló la arquitectura y se generaron los tres diagramas, de modo que el código se escribiera contra un contrato y no al revés. Después se implementaron los nodos de abajo hacia arriba: utilidades comunes, broker, trabajadores, coordinador. Luego se construyó el arnés de pruebas y

se ejecutó la línea base. Finalmente se inyectaron fallos, se identificaron cuellos de botella y se aplicaron ajustes, cada uno con una medición antes y después.

La guía sugiere un equipo de cuatro a cinco integrantes con roles diferenciados. Dado que la docente autorizó la modalidad de grupo reducido, el equipo de dos integrantes asumió los cinco roles con responsabilidades explícitas y sin solapamiento en las decisiones críticas: el líder y arquitecto tomó las decisiones de diseño y las registró; el desarrollador y evaluador implementó y midió; ambos documentaron y prepararon el video.

Tabla 4

Roles, responsabilidades y productos asociados

Rol	Responsable	Responsabilidades	Productos
Líder de proyecto	Juan Carlos Benítez Rojas	Planificación, cronograma, coordinación y entrega en LMS	Plan de proyecto, cronograma, checklist
Diseñador de arquitectura	Juan Carlos Benítez Rojas	Modelado UML, selección de patrones, decisiones de diseño	Tres diagramas, tabla de patrones
Desarrollador	Juan José Córdoba Eraso	Implementación de nodos, broker, coordinador y tablero	Código del prototipo
Evaluador de calidad	Juan José Córdoba Eraso	Pruebas de carga, caos, métricas y análisis de resultados	Resultados, gráficas, ajustes
Documentador y comunicador	Ambos	Informe APA 7, README, evidencias y video	Informe, repositorio, video

Tabla 5

Cronograma de ejecución del proyecto

Fecha	Fase	Actividades	Entregable
04/11/2026	Análisis	Alcance, roles, requisitos y supuestos	Plan de proyecto
05/11/2026	Diseño	Diagramas de componentes, secuencia y despliegue	Diagramas PlantUML
06/11/2026	Implementación I	Utilidades, broker y nodos trabajadores	Nodos operativos

Fecha	Fase	Actividades	Entregable
07/11/2026	Implementación II	Coordinador, Saga, Circuit Breaker y balanceo	Clúster funcional
08/11/2026	Observabilidad	Tablero, métricas y bitácora de eventos	Dashboard
09/11/2026	Evaluación	Carga, escalabilidad, caos y ajustes	Resultados y gráficas
10/11/2026	Documentación	Informe APA 7, README y repositorio	Informe y código
11/11/2026	Cierre	Video, revisión de coherencia y entrega	Video y entrega LMS

Tabla 6

Riesgos identificados y estrategias de mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Sobrecarga por concurrencia excesiva	Media	Alto	Pools acotados y backpressure en el broker
Nodo lento que degrade el pipeline	Alta	Alto	Circuit Breaker y estrategia de balanceo
Pérdida de trazabilidad de eventos	Media	Medio	Log JSONL inmutable y contadores de descarte
Tiempo insuficiente para el video	Media	Medio	Guion previo y demo ensayada del clúster

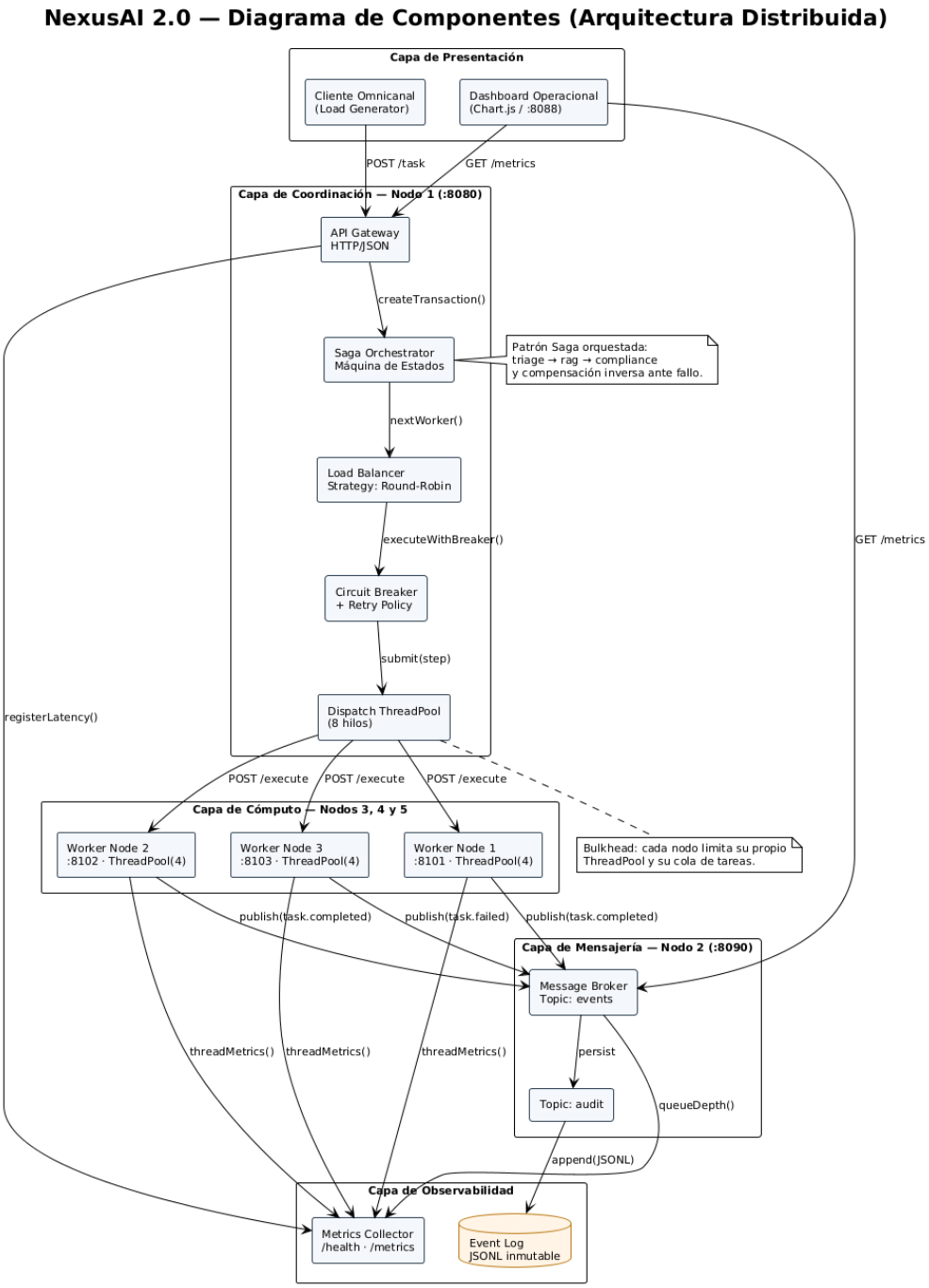
Modelado UML

El modelado se realizó con PlantUML, herramienta recomendada por la docente, lo que permitió versionar los diagramas como texto y regenerarlos con un script propio (PlantUML, 2024). Se elaboró un diagrama por categoría, conforme a la indicación de la guía: un diagrama de componentes, un diagrama de secuencia y un diagrama de despliegue. Los tres se incluyen a continuación con su explicación técnica.

Diagrama de componentes

Figura 1

Diagrama de componentes de la arquitectura distribuida de NexusAI 2.0



Nota. Elaboración propia con PlantUML. Las notas del diagrama indican el patrón aplicado en cada agrupación.

El diagrama organiza el sistema en cinco capas. La capa de presentación contiene al cliente omnicanal, que actúa como generador de carga, y al tablero operacional. La capa de coordinación concentra el API Gateway, el orquestador de la Saga, el balanceador, el Circuit Breaker y el pool de despacho. La capa de mensajería contiene el broker y sus topics. La capa de cómputo contiene los tres nodos trabajadores, cada uno con su propio ThreadPool. La capa de observabilidad contiene el recolector de métricas y el registro de eventos.

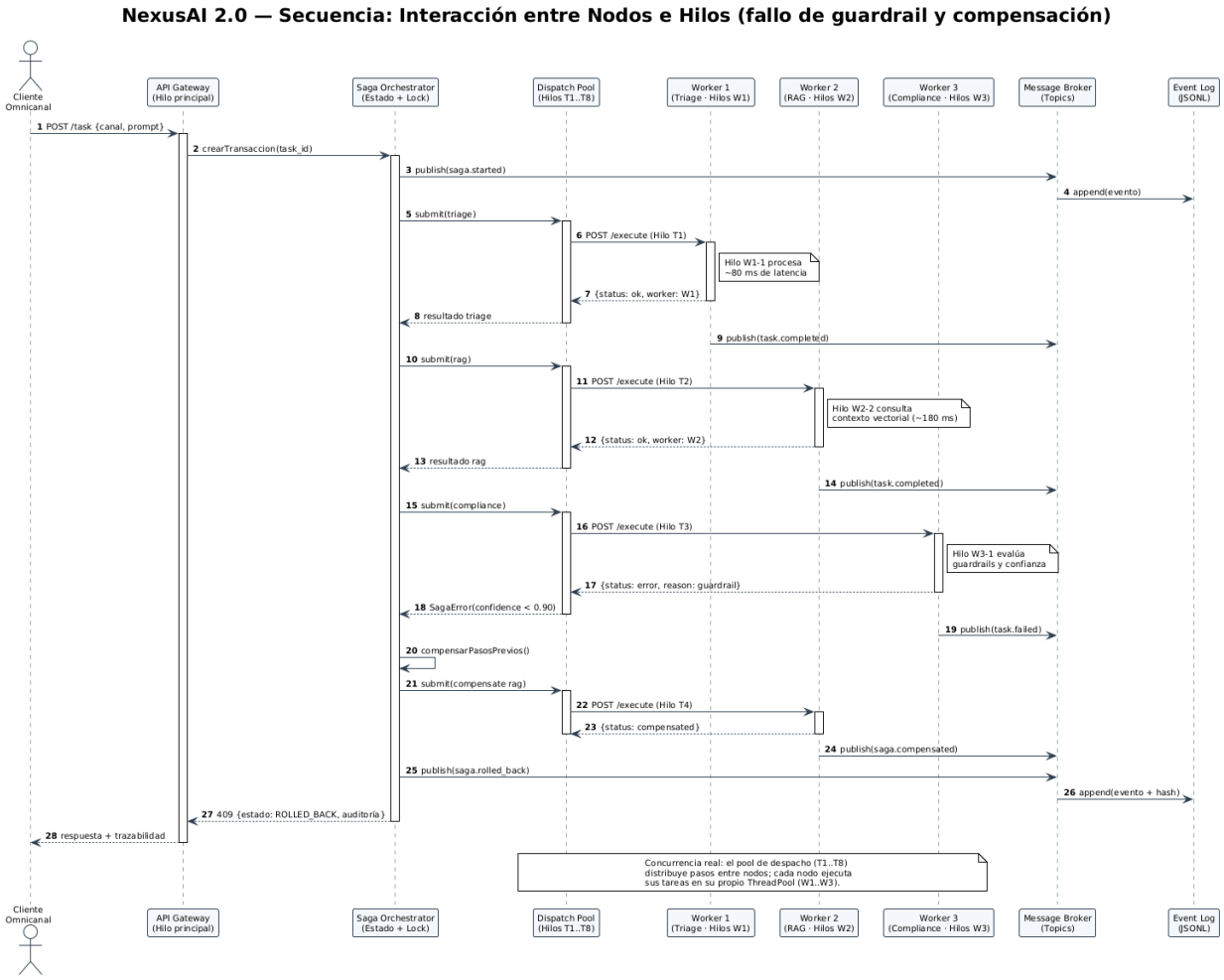
Las dependencias apuntan siempre hacia abajo y nunca en sentido inverso: el cliente depende del gateway, el gateway depende del orquestador, el orquestador depende del balanceador y del breaker, y el breaker depende del pool, que es el único componente que conoce las direcciones de los trabajadores. Los trabajadores, a su vez, dependen únicamente del broker, no del coordinador. Esta dirección única de dependencias es la que permite sustituir un trabajador por otro sin tocar la lógica de negocio y la que hace que el coordinador pueda ignorar cuántos nodos existen.

El diagrama evidencia dos decisiones arquitectónicas. La primera es que el coordinador no ejecuta cómputo: solo orquesta. Esto separa la escalabilidad de la coordinación, porque agregar workers no requiere replicar el orquestador ni duplicar el estado de las Sagas. La segunda es que el broker es el único punto de publicación de eventos, lo que convierte a la observabilidad en un consumidor más y evita que los nodos escriban directamente en la bitácora.

Diagrama de secuencia

Figura 2

Diagrama de secuencia con interacción entre nodos e hilos y compensación de la Saga



Nota. Elaboración propia con PlantUML. Los hilos T1 a T8 pertenecen al pool de despacho; los hilos W1 a W3 pertenecen a cada nodo trabajador.

El diagrama representa una transacción que falla en la etapa de cumplimiento y obliga a compensar los pasos anteriores. Las líneas de vida distinguen explícitamente los hilos: el pool de despacho aparece como un participante con hilos T1 a T8, y cada trabajador aparece con su propio pool de hilos W1 a W3. Esta separación es la que exige la guía cuando pide representar la

interacción entre nodos e hilos, y es la que permite leer el diagrama como un modelo de concurrencia y no solo como una secuencia de llamadas.

La secuencia comienza cuando el cliente envía la solicitud al gateway, que delega en el orquestador la creación de la transacción. El orquestador publica el evento de inicio y solicita al pool el primer paso. El pool asigna un hilo de despacho y este invoca al trabajador correspondiente. Cada trabajador ejecuta el paso en su propio hilo, responde y publica el evento de finalización. El proceso se repite para la recuperación de contexto y para el cumplimiento.

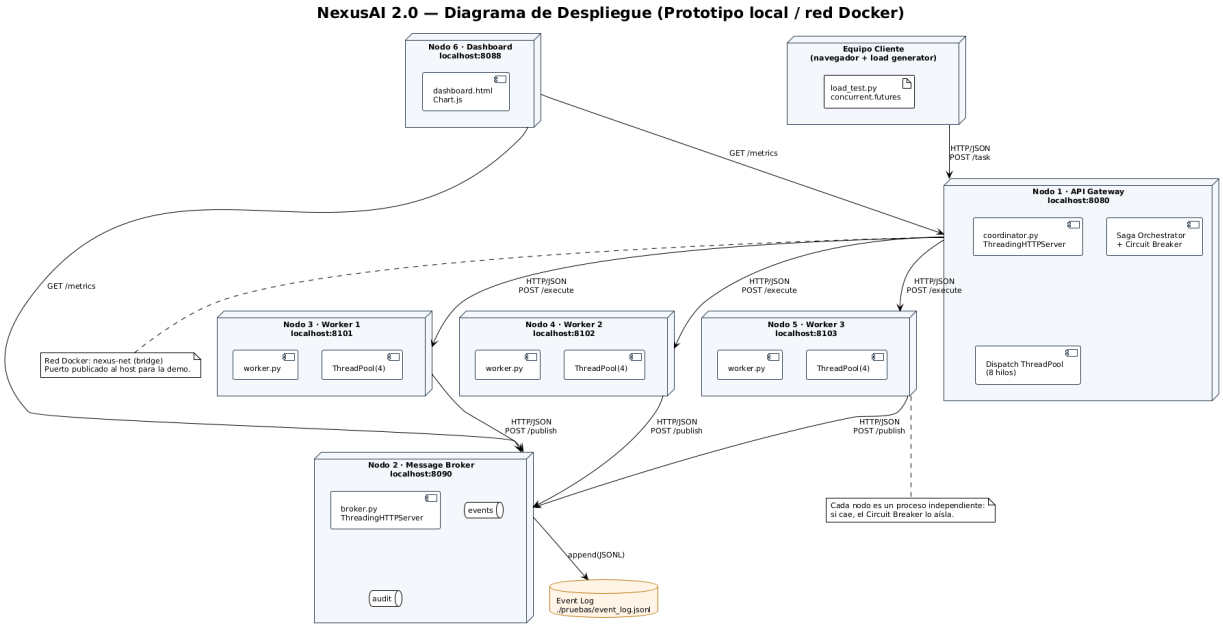
Cuando el trabajador de cumplimiento detecta que la confianza semántica no alcanza el umbral, devuelve un error de negocio. El orquestador no reintenta ese paso, porque reintentarlo produciría el mismo resultado; en su lugar, inicia la compensación inversa. Primero compensa la recuperación de contexto y luego la clasificación. Cada compensación se envía preferentemente al mismo nodo que ejecutó el paso original, lo que preserva la afinidad semántica. Finalmente, el orquestador publica el evento de reversión y responde al cliente con un código 409 que incluye la trazabilidad completa.

El diagrama muestra también la asincronía de los eventos: las flechas hacia el broker no bloquean la respuesta al cliente, y la bitácora recibe los eventos en el orden en que se publican. Esta decisión se refleja en el código con una cola de eventos y un hilo publicador por nodo.

Diagrama de despliegue

Figura 3

Diagrama de despliegue del prototipo en red local y su equivalencia con Docker



Nota. Elaboración propia con PlantUML. Cada nodo corresponde a un proceso independiente con su propio puerto.

El diagrama describe la distribución física y lógica del sistema. Se identifican seis nodos: el equipo cliente, que ejecuta el navegador y el generador de carga; el nodo del API Gateway en el puerto 8080; el nodo del broker en el puerto 8090; los tres nodos trabajadores en los puertos 8101, 8102 y 8103; y el nodo del tablero en el puerto 8088. El registro de eventos se representa como un artefacto de almacenamiento compartido por el broker.

Las conexiones indican el protocolo y el sentido de la comunicación. El cliente se conecta al gateway por HTTP/JSON; el gateway se conecta a cada trabajador por HTTP/JSON; cada trabajador publica en el broker; el broker escribe en el registro; el tablero consulta las métricas del coordinador y del broker. No hay conexiones directas entre trabajadores, lo que confirma el aislamiento.

El diagrama incluye una nota sobre la red bridge de Docker. Aunque la ejecución se realizó con procesos locales para simplificar la demostración, la topología es equivalente: cada nodo podría ejecutarse en un contenedor conectado a una red privada, publicando solo los puertos necesarios al host. Esta equivalencia es importante porque demuestra que la arquitectura no depende del mecanismo de despliegue, sino de los contratos entre nodos.

Selección y justificación de patrones de diseño

La selección de patrones se realizó a partir de los problemas identificados en el análisis: coordinación de transacciones largas, desacoplamiento de la comunicación, aislamiento de fallos, distribución de carga y observabilidad. Para cada patrón se documenta el problema que resuelve, la forma en que se implementó y el compromiso que introduce, siguiendo el criterio de que un patrón no es una receta, sino una decisión con costos.

Saga orquestada

El problema es la consistencia de una transacción que abarca varios servicios sin transacciones distribuidas. Aplicar un protocolo de confirmación en dos fases bloquearía recursos durante la latencia no determinista de un modelo de lenguaje y expondría al sistema a interbloqueos. La Saga, propuesta originalmente por Garcia-Molina y Salem (1987), resuelve el problema descomponiendo la transacción en pasos locales y definiendo una acción compensatoria para cada paso.

En NexusAI 2.0 la Saga se implementó con orquestación centralizada: el coordinador mantiene la máquina de estados y decide el orden de los pasos y las compensaciones. Se eligió orquestación y no coreografía porque el equipo necesita visibilidad central del estado de cada transacción y porque la compensación debe ejecutarse en orden inverso, lo que es más difícil de garantizar cuando cada servicio reacciona a eventos sin un coordinador (Richardson, 2018). El

costo es que el orquestador se convierte en un componente crítico; se mitiga manteniéndolo sin estado durable y permitiendo que las Sagas en vuelo se pierdan de forma controlada si el coordinador falla, una limitación aceptada para el alcance del prototipo.

Broker de mensajes

El problema es el acoplamiento temporal entre quien produce un evento y quien lo consume. Si un trabajador escribiera directamente en la bitácora, una caída del almacenamiento bloquearía la respuesta al usuario. El patrón Broker, descrito por Hohpe y Woolf (2003), introduce un intermediario que recibe los mensajes y los pone a disposición de los consumidores de forma desacoplada.

El broker del prototipo mantiene topics en memoria, publica cada evento en un archivo JSONL y expone métricas de profundidad y de descarte. Se eligió un broker propio y no Kafka porque el objetivo es demostrar el patrón y no la tecnología, y porque la biblioteca estándar permite reproducir el experimento sin infraestructura adicional. El compromiso es que el broker es un punto único de fallo y que su cola es acotada; se mitiga con una política de descarte del evento más antiguo cuando la cola alcanza diez mil elementos, de modo que el productor nunca se bloquee.

Mediator

El problema es la explosión de dependencias cuando muchos componentes necesitan comunicarse entre sí. Si cada trabajador conociera a los demás y al tablero, agregar un nodo obligaría a modificar múltiples módulos. El patrón Mediator, documentado por Gamma et al. (1994), concentra la comunicación en un intermediario que conoce a los participantes y evita que estos se conozcan entre sí.

En el prototipo, el coordinador actúa como mediador: los trabajadores solo conocen su propio contrato y el broker; no saben cuántos nodos existen ni quién los invoca. Esta decisión reduce el acoplamiento y hace que el sistema sea extensible: agregar un cuarto trabajador requiere registrarlo en el balanceador, no modificar a los demás. El costo es que el mediador debe conocer la topología, lo que se resuelve con el endpoint de administración de trabajadores.

Proxy y API Gateway

El problema es exponer una interfaz única y controlada a los clientes cuando el sistema está compuesto por múltiples servicios. El patrón Proxy, en su variante de API Gateway, interpone un componente que recibe las solicitudes, aplica políticas transversales y las enruta al servicio correspondiente (Fowler, 2014).

El gateway del prototipo valida la estructura de la solicitud, aplica el control de concurrencia mediante el pool de despacho y traduce el resultado de la Saga a un código HTTP significativo: 200 para completada, 409 para compensada y error controlado para indisponibilidad. Esta traducción es valiosa porque el cliente no necesita conocer la topología interna ni distinguir entre un fallo de negocio y un fallo de infraestructura. El costo es que el gateway se convierte en el punto de entrada único y debe escalarse con cuidado; en el prototipo se resuelve con un pool de dieciséis hilos.

Circuit Breaker

El problema es el fallo en cascada: cuando un nodo se degrada, los reintentos y las esperas consumen hilos y propagan la lentitud a todo el sistema. El Circuit Breaker, popularizado por Nygard (2018), monitorea los fallos y abre el circuito para dejar de enviar tráfico al componente defectuoso durante un periodo de enfriamiento.

La implementación del prototipo tiene tres estados. En CLOSED el tráfico fluye con normalidad. Tras tres fallos consecutivos el circuito pasa a OPEN y el nodo se excluye del balanceo durante cinco segundos. Al cumplirse el enfriamiento pasa a HALF_OPEN y permite una petición de prueba; si tiene éxito vuelve a CLOSED y si falla regresa a OPEN. El patrón se aplica por nodo y por paso, de modo que un fallo en el cumplimiento no bloquea la clasificación. El costo es que durante el enfriamiento el sistema opera con menos capacidad; los resultados muestran que esa pérdida es preferible a la degradación total.

Bulkhead

El problema es que un recurso compartido agotado consume la capacidad de todo el sistema. El patrón Bulkhead, tomado de la ingeniería naval, divide los recursos en compartimentos estancos para que la inundación de uno no hunda el barco. En software, se implementa con pools de hilos o de conexiones separados por consumidor.

Cada trabajador del prototipo tiene su propio ThreadPool de cuatro hilos. Si un paso se vuelve lento, solo se agotan los hilos de ese trabajador; los demás nodos siguen atendiendo. El coordinador también aplica el patrón con un pool de despacho acotado, que impide que una avalancha de clientes consuma memoria sin límite. El costo es que la capacidad máxima queda determinada por la suma de los pools, lo que exige dimensionarlos con base en la carga esperada.

Observer distribuido

El problema es notificar cambios de estado a múltiples interesados sin acoplar al emisor con cada receptor. El patrón Observer define una relación de uno a muchos en la que los observadores se suscriben a un sujeto y reciben notificaciones cuando este cambia.

En un sistema distribuido, el sujeto es el broker y los observadores son los consumidores de cada topic. El prototipo publica eventos como saga.started, saga.step.completed,

task.completed, task.failed, saga.compensated y saga.rolled_back. El tablero y la bitácora pueden consumirlos sin que los nodos sepan quién los escucha. Esta variante distribuida del patrón es la que permite agregar un nuevo consumidor —por ejemplo, un sistema de alertas— sin modificar a los productores.

Strategy

El problema es que la mejor política de balanceo depende del contexto: round-robin es simple y equitativo cuando los nodos son homogéneos, pero least-connections es mejor cuando las tareas tienen duración variable. El patrón Strategy, descrito por Gamma et al. (1994), encapsula algoritmos intercambiables detrás de una interfaz común.

El balanceador del prototipo implementa dos estrategias: RoundRobinStrategy, que asigna en orden circular, y LeastConnectionsStrategy, que elige el nodo con menos peticiones en vuelo. La estrategia se selecciona al iniciar el coordinador, sin modificar el resto del código. Esta decisión demuestra que la variabilidad del algoritmo no debe filtrarse a la lógica de la Saga.

Retry

El problema es que muchos fallos distribuidos son transitorios: un paquete perdido, una conexión reiniciada o un nodo que se recupera. El patrón Retry reintenta la operación, preferiblemente en un recurso distinto, antes de declarar el fallo.

El coordinador implementa dos intentos por paso, con la restricción de que el segundo intento se dirige a un nodo que no haya fallado. La decisión clave es distinguir entre errores de nodo y errores de negocio: un error de guardrail no se reintenta, porque el resultado sería idéntico y se desperdiciarían recursos. Esta distinción se materializa en el campo kind de la respuesta del trabajador y es una de las lecciones más importantes del ejercicio.

Síntesis de patrones

Tabla 7

Correspondencia entre problemas, patrones y evidencia en el prototipo

Patrón	Problema que resuelve	Implementación	Evidencia
Saga orquestada	Consistencia sin transacciones distribuidas	SagaOrchestrator y compensación inversa	31 Sagas compensadas sin errores
Broker	Acoplamiento temporal productor-consumidor	broker.py con topics y JSONL	9 735 eventos consumidos sin descartes
Mediator	Dependencias cruzadas entre nodos	Coordinador como único intermediario	Nodos sin conocimiento mutuo
Proxy / API Gateway	Interfaz única y políticas transversales	CoordinatorHandler y POST /task	Contrato 200 / 409 / error
Circuit Breaker	Fallo en cascada por nodo degradado	Tres estados con enfriamiento de 5 s	w2 aislado con el circuito OPEN
Bulkhead	Agotamiento de recursos compartidos	ThreadPool por nodo y pool de despacho	Aislamiento de latencia en w2
Observer distribuido	Notificación desacoplada de cambios	Eventos publicados en el broker	Tablero y bitácora desacoplados
Strategy	Algoritmo de balanceo variable	RoundRobin y LeastConnections	Estrategia configurable al iniciar
Retry	Fallos transitorios de red	Dos intentos en nodos distintos	100 % atendido con w3 caído

Implementación y simulación

El diseño se materializó en un clúster de seis procesos independientes escrito en Python 3.14.7, sin dependencias externas para la ejecución del sistema. El código suma 2 111 líneas de Python distribuidas en nueve módulos y un tablero HTML de 16 KB. La implementación respetó la separación de responsabilidades definida en el modelado: ningún módulo concentra más de una responsabilidad arquitectónica y el único código compartido es un conjunto de utilidades HTTP y estadísticas, no un estado común. Esta correspondencia entre diseño e implementación es la que permite afirmar que los diagramas describen el sistema real y no una intención paralela (Sommerville, 2016).

Estructura del código

Tabla 8

Módulos del prototipo, responsabilidad y tamaño

Módulo	Responsabilidad	Patrón asociado	Líneas
`common.py`	Servidor JSON, cliente HTTP, percentiles, esperas activas	Utilidades transversales	138
`broker.py`	Topics en memoria, bitácora JSONL, consumidor de auditoría	Broker y Observer	166
`worker.py`	Nodo de cómputo con ThreadPool, caos y métricas	Bulkhead	239
`coordinator.py`	Gateway, Saga, breaker, balanceo, métricas	Saga, Proxy, Strategy, Retry	585
`cluster.py`	Ciclo de vida de los procesos del clúster	Orquestación de despliegue	226
`load_test.py`	Clientes concurrentes y percentiles	Generador de carga	183
`chaos_test.py`	Inyección de latencia, fallos y apagado de nodos	Ingeniería del caos	90
`benchmark.py`	Suite de escenarios, artefactos y gráficas	Evaluación experimental	422
`run_cluster.py`	Demo interactiva del clúster	Operación	62
`dashboard.html`	Consola de observabilidad en tiempo real	Observabilidad	—

La lectura de la tabla revela una decisión deliberada: el coordinador es el módulo más extenso porque concentra las políticas transversales, mientras que los trabajadores son pequeños y homogéneos. Esa asimetría es correcta en una arquitectura de orquestación centralizada: la complejidad se paga una vez en el coordinador y no se replica en cada nodo. Si el sistema evolucionara a coreografía, la balanza se invertiría y cada trabajador tendría que conocer el siguiente paso, con el costo de acoplamiento que ello implica (Newman, 2021).

Modelo de procesos, hilos y sincronización

El prototipo distingue cuatro niveles de concurrencia que se ejecutan simultáneamente y que el diagrama de secuencia representa de forma explícita. En el nivel de proceso, cada nodo es un proceso del sistema operativo con memoria independiente: un fallo de memoria, una excepción no capturada o un bloqueo de un trabajador no derriba a los demás. En el nivel de servidor, cada nodo usa `ThreadingHTTPServer`, que crea un hilo por conexión y permite atender varias peticiones simultáneas sin bloquear al aceptador. En el nivel de despacho, el coordinador mantiene un pool de dieciséis hilos que ejecutan Sagas completas. En el nivel de cómputo, cada trabajador mantiene un pool de cuatro hilos que ejecuta los pasos.

Tabla 9

Hilos por proceso y función en el prototipo

Proceso	Hilos	Función
Coordinador	1 principal, 16 de despacho, 1 publicador, 1 por conexión	Aceptar Sagas, orquestar pasos, publicar eventos
Trabajador	1 principal, 4 de cómputo, 1 publicador, 1 por conexión	Ejecutar pasos y publicar resultados
Broker	1 principal, 1 consumidor de auditoría, 1 por conexión	Encolar, drenar y auditar eventos
Tablero	1 principal, 1 por conexión	Servir el HTML y las consultas del navegador

La sincronización se resuelve con candados de grano fino y estructuras thread-safe. El broker protege sus colas y contadores con un lock y usa `queue.Queue` acotada a diez mil elementos. El coordinador protege la lista de trabajadores, los contadores de vuelo y las métricas con un lock propio. Cada trabajador protege sus contadores y su ventana de latencias con un lock independiente. No existe memoria compartida entre procesos, de modo que el sistema es un ejemplo de arquitectura *shared-nothing*: la coordinación se hace por mensajes y no por estado compartido (Kleppmann, 2017).

Una precisión técnica importante: las etapas cognitivas simuladas usan `time.sleep`, que libera el GIL de Python y permite que los cuatro hilos de un trabajador se solapen realmente. Si en producción las etapas fueran cómputo intensivo en Python puro, el GIL limitaría la concurrencia y sería necesario migrar a procesos, extensiones nativas o servicios externos. El prototipo es honesto en ese límite y lo documenta como trabajo futuro.

Contrato HTTP y flujo de una transacción

El contrato es uniforme en todos los nodos: JSON sobre HTTP, con códigos de estado que distinguen el resultado de negocio del resultado de infraestructura. Una transacción se inicia con una petición al gateway y termina con una respuesta que contiene la trazabilidad completa de los pasos.

```
POST /task
{
  "payload": {
    "prompt": "escalar incidente critico #42",
    "canal": "whatsapp"
  }
}

HTTP/1.1 200 OK
{
  "status": "completed",
  "saga_id": "e77feb53a1",
  "steps": [
    {"step": "triage",     "worker": "w1", "attempt": 1, "latency_ms": 118.4},
    {"step": "rag",        "worker": "w2", "attempt": 1, "latency_ms": 231.7},
    {"step": "compliance", "worker": "w2", "attempt": 1, "latency_ms": 134.9}
  ],
  "duration_ms": 501.8,
  "e2e_latency_ms": 508.2
}
```

Cuando la etapa de cumplimiento rechaza la transacción por una regla de negocio, el gateway responde 409 y el cuerpo incluye el paso fallido y la compensación ejecutada. Cuando ningún nodo está disponible porque todos los circuitos están abiertos, responde con un error controlado. La distinción se materializa en el campo `kind`: `business` indica que el nodo respondió correctamente y la regla de negocio falló; `node` indica que el nodo no pudo ejecutar el

paso. Esta diferencia, que parece menor, es la que evita reintentos inútiles y falsos positivos en el Circuit Breaker.

Simulación de las etapas cognitivas

Las etapas se simularon con latencias base y una componente aleatoria uniforme de hasta treinta milisegundos, con el fin de reproducir la variabilidad de un servicio cognitivo real sin introducir el costo ni la no determinación de un modelo de lenguaje.

Tabla 10

Latencias simuladas por etapa del pipeline

Etapa	Latencia base	Jitter	Compensación	Justificación
Triage	80 ms	0–30 ms	50 ms	Clasificación de intención, la etapa más liviana
RAG	180 ms	0–30 ms	50 ms	Recuperación de contexto, la etapa más costosa
Compliance	120 ms	0–30 ms	50 ms	Validación de guardrails, costo intermedio
Compensate	50 ms	0–30 ms	—	Reversión idempotente del paso

La decisión de simular y no invocar un modelo real se justifica por tres razones. Primero, la reproducibilidad: un modelo con temperatura mayor que cero produce latencias y respuestas distintas en cada ejecución, lo que impediría comparar escenarios. Segundo, el costo y la cuota: un experimento de mil doscientas transacciones con tres pasos cada una superaría los límites de una cuenta académica. Tercero, el objetivo del ejercicio es evaluar la arquitectura y no la calidad del modelo. La latencia simulada es, por tanto, una variable controlada y no una medida del mundo real; los resultados deben leerse como tendencias arquitectónicas y no como un benchmark de inteligencia artificial.

Puesta en marcha y reproducibilidad

El clúster se levanta con un comando y se detiene con la interrupción del proceso. El script de orquestación espera activamente a que cada nodo responda antes de continuar, de modo que el orden de arranque sea determinista.

```
python run_cluster.py --workers 3 --threads 16 --worker-threads 4

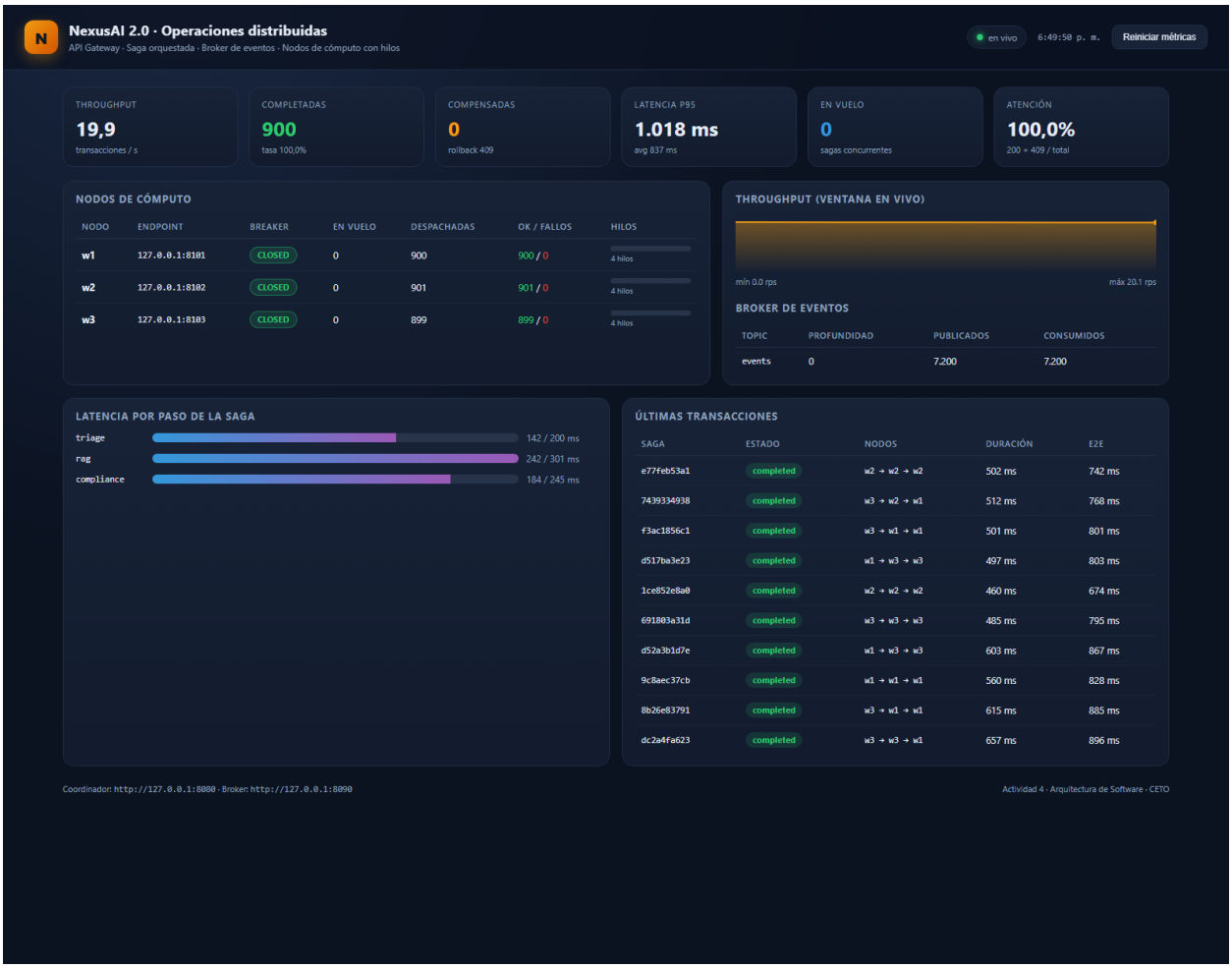

python load test.py --tasks 120 --concurrency 24 --label carga manual
python chaos_test.py latency --worker http://127.0.0.1:8102 --ms 300
python chaos_test.py fail    --worker http://127.0.0.1:8102 --rate 1.0
python chaos test.py reset
python benchmark.py
```

El tablero queda disponible en `http://127.0.0.1:8088/dashboard.html`. La suite completa genera `resultados.json`, `resumen.md`, la bitácora `event_log.jsonl` y las gráficas en `pruebas/graficas/`. Todos los experimentos de este informe se ejecutaron con el mismo comando y la misma configuración, lo que permite reproducirlos y auditarlos.

Observabilidad y trazabilidad

Figura 4

Tablero operacional de NexusAI 2.0 durante una ejecución de carga



Nota. Elaboración propia. El tablero consulta cada 1,5 segundos las métricas del coordinador y del broker. La captura corresponde a una corrida de 900 transacciones con la estrategia least-connections.

El tablero resume el estado del sistema en seis indicadores: throughput, transacciones completadas, transacciones compensadas, latencia p95, Sagas en vuelo y tasa de atención. Debajo presenta la tabla de nodos con su estado de Circuit Breaker, peticiones en vuelo, despachadas y relación entre éxitos y fallos; la tabla del broker con profundidad de cola, publicados y consumidos; la latencia por paso de la Saga; y las últimas transacciones con su

identificador, estado, nodos participantes y duración. Esta vista es la que permite diagnosticar un escenario de caos en tiempo real y no después de leer un archivo de log.

La trazabilidad se apoya en eventos publicados en el broker. Cada transición produce un evento con identificador de Saga, nodo responsable y latencia, y el broker los persiste en un archivo JSONL inmutable que sirve como bitácora de auditoría.

Tabla 11

Eventos publicados durante una transacción

Evento	Emisor	Información principal
`saga.started`	Coordinador	Identificador de Saga y canal de origen
`saga.step.completed`	Coordinador	Paso, nodo, intento y latencia
`task.completed`	Trabajador	Paso ejecutado y latencia local
`task.failed`	Trabajador	Paso, motivo y tipo de fallo
`saga.completed`	Coordinador	Duración total de la Saga
`saga.rolled_back`	Coordinador	Paso fallido, motivo y pasos compensados


```
{"ts": 1794267910808.94, "topic": "events", "message": {"event": "saga.started",
 "source": "coordinator", "ts": 1794267910757.16, "saga_id": "ddefa5c333", "canal":
"web"}}
{"ts": 1794267974934.96, "topic": "events", "message": {"event": "saga.rolled_back",
 "source": "coordinator", "ts": 1794267974848.66, "saga_id": "7bdadaa043",
 "failed_step": "compliance", "reason": "guardrail: confianza semantica < 0.90",
 "compensated_steps": 2, "duration_ms": 673.93}}
{"ts": 1794267998822.08, "topic": "events", "message": {"event": "saga.completed",
 "source": "coordinator", "ts": 1794267998812.62, "saga_id": "9b8b797e4e",
 "duration_ms": 482.17}}
```


La bitácora de la suite acumuló 9 735 eventos consumidos por el suscriptor de auditoría, sin descartes y con una profundidad de cola de cero al final de cada escenario. Ese resultado no fue inmediato: en la primera versión el broker publicaba sin consumidor y la cola crecía hasta rozar el límite de diez mil elementos, hallazgo que se documenta como ajuste en la sección siguiente.

Evaluación y ajustes

La evaluación se diseñó como un experimento controlado de una sola variable a la vez: mismo equipo, misma carga, misma configuración y un cambio por escenario. El objetivo no fue

obtener el número más alto posible, sino establecer relaciones causales defendibles entre una decisión de arquitectura y un efecto medible en el throughput, la latencia o la disponibilidad. Este enfoque corresponde a la evaluación de atributos de calidad mediante escenarios propuestos por Bass et al. (2022) y se alinea con las categorías de eficiencia de desempeño y fiabilidad de ISO/IEC 25010 (International Organization for Standardization, 2011).

Metodología experimental

Tabla 12

Configuración del entorno y de la carga de prueba

Parámetro	Valor
Equipo	Windows 11, 4 núcleos lógicos, Python 3.14.7
Nodos de cómputo	3 procesos independientes, 4 hilos cada uno
Hilos de despacho del coordinador	16
Estrategia de la suite principal	round-robin (peor caso)
Carga de escalabilidad	180 transacciones, concurrencia 24
Carga de robustez	120 transacciones, concurrencia 24
Calentamiento	12 transacciones antes del primer escenario
Semilla	Derivada del nombre del escenario
Reinicio de métricas	Antes de cada escenario, con breakers reiniciados
Total de transacciones	1 200 en la suite, 0 errores de sistema

Se distingue entre transacción atendida y transacción completada. Una transacción atendida es aquella que recibió una respuesta de negocio, sea 200 o 409; una transacción completada es la que terminó con 200. La tasa de atención es la métrica de disponibilidad, porque una compensación correcta es una respuesta válida del sistema y no un error. La tasa de completitud es la métrica de éxito funcional. Esta distinción evita el error frecuente de reportar una compensación como fallo y penalizar injustamente a la arquitectura.

Los percentiles se calcularon sobre la muestra completa de latencias extremo a extremo medidas por el cliente, no sobre promedios de promedios. El throughput se calculó como

transacciones atendidas divididas por el tiempo de pared del escenario. Los contadores del broker son acumulativos desde el arranque del proceso, mientras que los del coordinador se reinician en cada escenario; por eso las cifras de eventos se reportan al final de la suite y las de latencia por escenario.

Escalabilidad horizontal

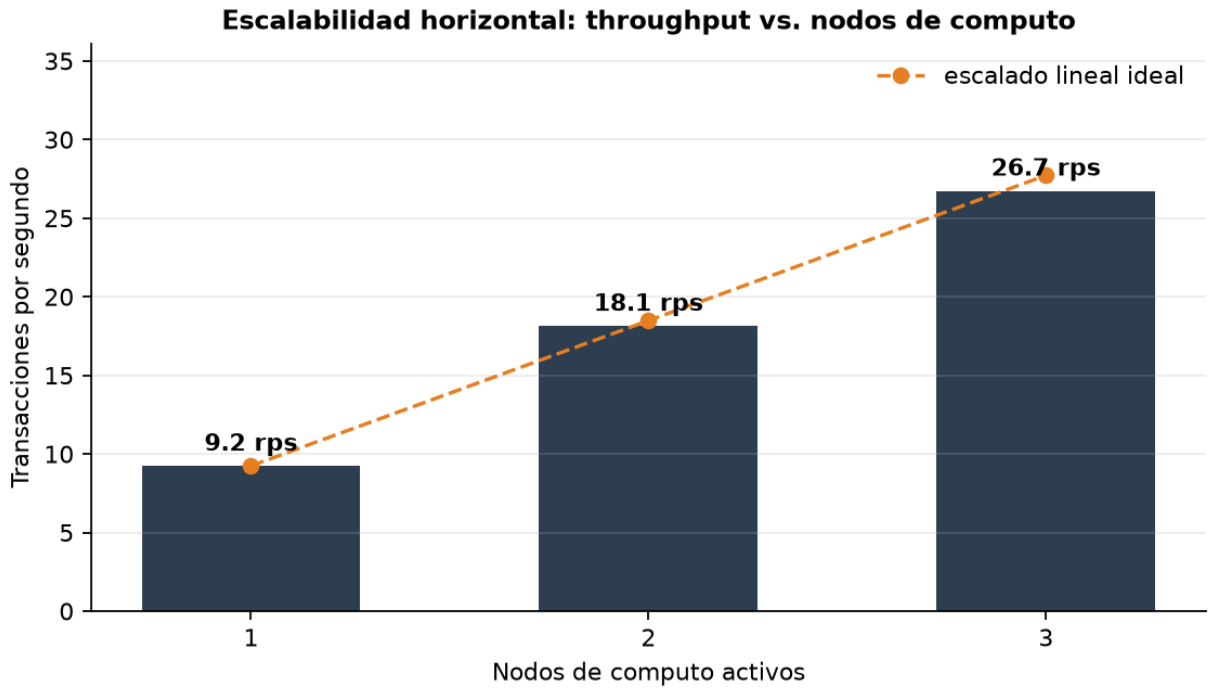
Tabla 13

Escalabilidad horizontal con carga constante de 180 transacciones

Nodos	Throughput (rps)	Speedup	Eficiencia	p50 (ms)	p95 (ms)	p99 (ms)
1	9,24	1,00×	100,0 %	2 131,3	3 185,0	3 243,4
2	18,14	1,96×	98,2 %	1 047,7	1 617,0	1 638,8
3	26,74	2,89×	96,5 %	854,2	1 043,4	1 127,0

Figura 5

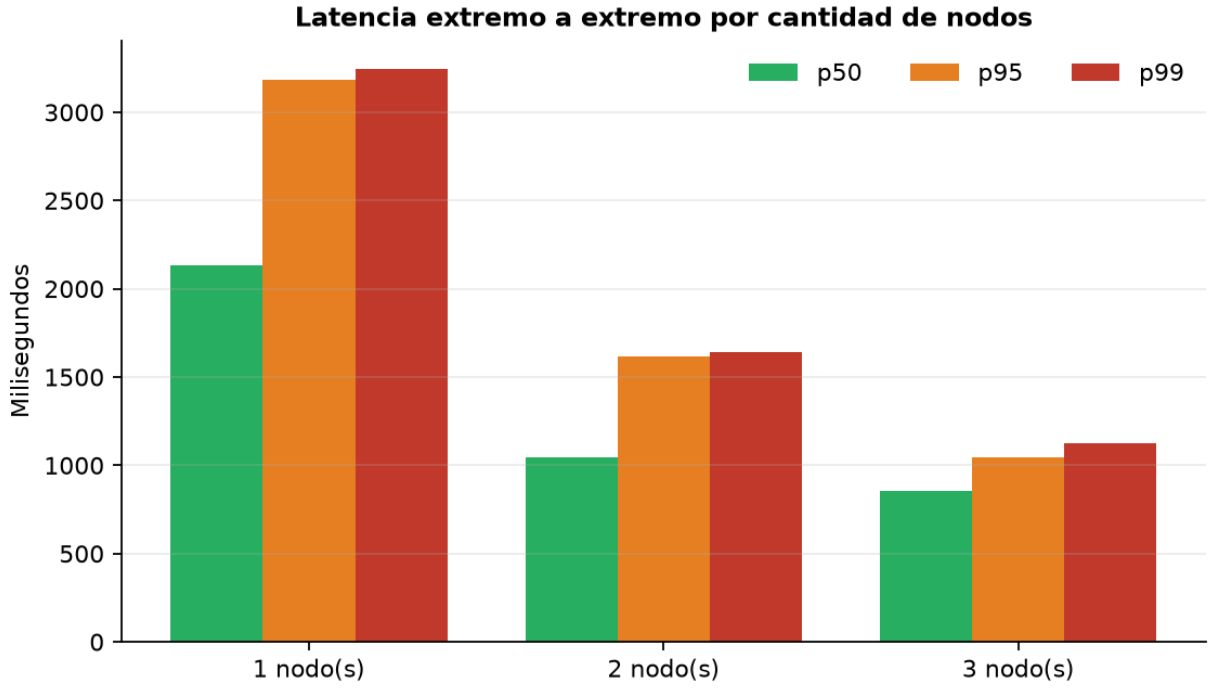
Throughput frente al número de nodos de cómputo



Nota. Elaboración propia. La línea punteada representa el escalado lineal ideal a partir del rendimiento de un nodo.

Figura 6

Latencia extremo a extremo por cantidad de nodos



Nota. Elaboración propia. p50, p95 y p99 medidos sobre 180 transacciones por escenario.

El resultado principal es un escalado casi lineal: el throughput se multiplica por 1,96 al duplicar los nodos y por 2,89 al triplicarlos, con una eficiencia del 98,2 % y del 96,5 %. La latencia acompaña la tendencia: el p95 cae de 3 185 ms con un nodo a 1 043 ms con tres, una reducción de 3,05 veces, y el p50 cae de 2 131 ms a 854 ms. El costo por paso confirma el mecanismo: la recuperación de contexto pasa de 626 ms a 248 ms de promedio y la clasificación de 433 ms a 146 ms.

La leve pérdida de eficiencia al llegar a tres nodos tiene una explicación física y no un defecto de diseño. El equipo de pruebas tiene cuatro núcleos lógicos y en la misma máquina conviven el coordinador, tres trabajadores, el broker, el tablero y veinticuatro clientes. Con tres trabajadores hay doce hilos de cómputo, dieciséis de despacho y veinticuatro clientes

compitiendo por cuatro núcleos. El sistema sigue escalando porque las etapas simuladas duermen y liberan el procesador, pero la contención de CPU empieza a notarse. En un despliegue real, con nodos en máquinas distintas, la eficiencia debería ser aún mayor. El límite teórico lo impone el coordinador, que es un punto único de despacho: aunque los trabajadores crezcan sin límite, el pool de dieciséis hilos y la serialización JSON acotan la tasa máxima. Ese es el componente que habría que replicar primero en una siguiente iteración.

Robustez ante fallos

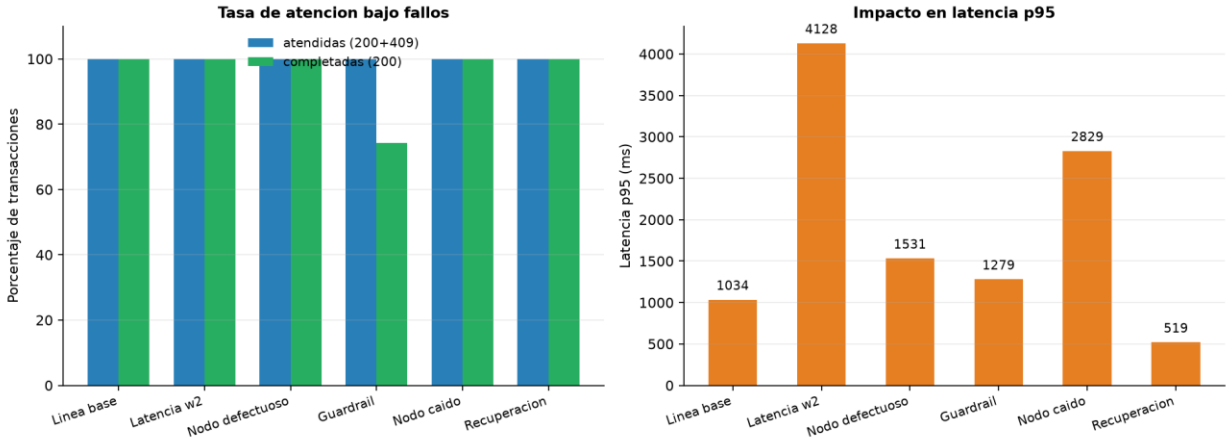
Tabla 14

Comportamiento del sistema en seis escenarios de fallo

Escenario	Tareas	Atendidas	Completadas	Compensadas	Throughput (rps)	p95 (ms)
Línea base	120	100 %	120	0	26,37	1 033,6
Latencia +300 ms en w2	120	100 %	120	0	8,83	4 128,4
w2 falla el 100 %	120	100 %	120	0	17,87	1 531,2
Guardrail, 25 % rechazos	120	100 %	89	31	23,82	1 279,2
w3 apagado	120	100 %	120	0	14,55	2 828,8
w3 recuperado	60	100 %	60	0	23,53	518,6

Figura 7

Tasa de atención y latencia p95 bajo distintos fallos



Nota. Elaboración propia. La barra azul representa transacciones atendidas (200 o 409) y la verde, transacciones completadas (200).

La primera conclusión de robustez es contundente: en los seis escenarios se atendió el cien por ciento de las transacciones y no se produjo un solo error de sistema en las 1 200 transacciones de la suite. El sistema nunca dejó a un cliente sin respuesta de negocio. La segunda conclusión es que la disponibilidad no es gratis: cada fallo se paga en throughput o en latencia, y el costo depende de qué patrón se activa.

Con latencia inyectada en w2, el throughput cae un 66,5 % y el p95 se multiplica por cuatro. El Circuit Breaker no se abre porque las llamadas no fallan: tardan trescientos milisegundos más y terminan bien. El daño no viene del error sino de la ocupación: con round-robin, un tercio de los pasos llega al nodo lento y cada llamada retiene un hilo de despacho durante más tiempo, hasta saturar los dieciséis hilos del coordinador. Este hallazgo es el más importante del ejercicio porque muestra que el Bulkhead por trabajador no protege al coordinador: el recurso escaso es el pool de despacho compartido. La solución no es un breaker,

porque no hay fallo, sino una estrategia de balanceo consciente de la carga, que se evalúa como ajuste más adelante.

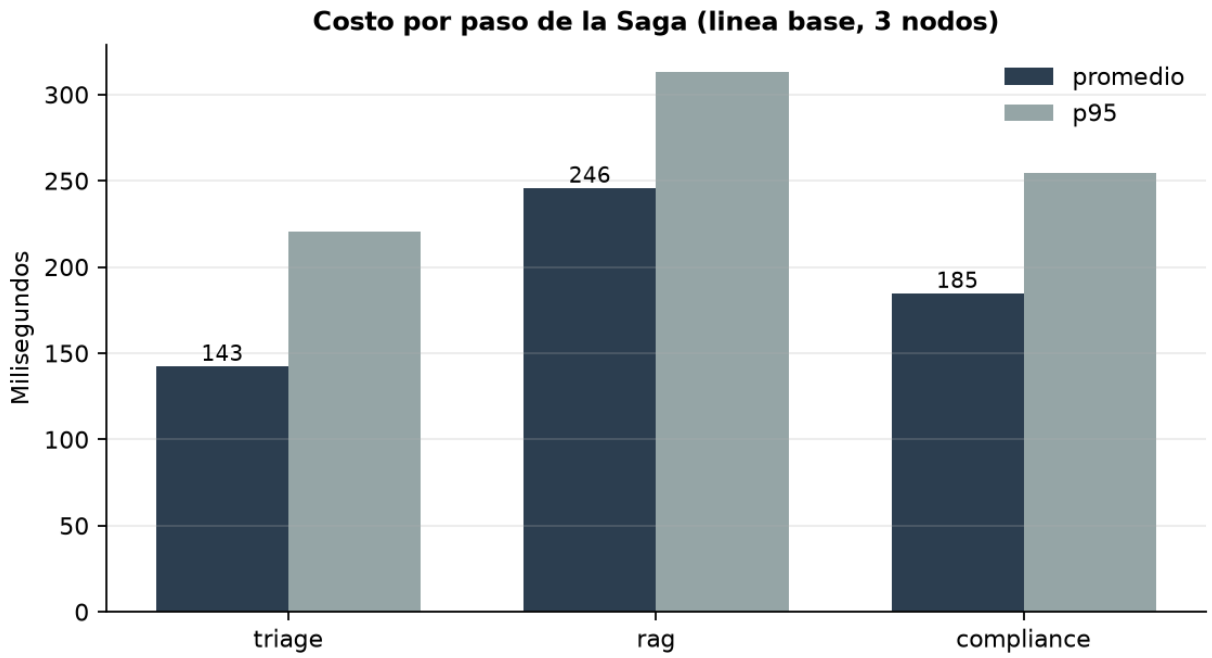
Cuando w2 falla el cien por ciento de sus peticiones, el sistema reacciona como estaba diseñado: el reintento envía el paso a otro nodo y, tras nueve fallos, el circuito de w2 pasa a OPEN. El throughput baja al 67,8 % de la línea base, pero todas las transacciones se completan. Es el comportamiento esperado de Retry más Circuit Breaker: se pierde capacidad, no servicio. Cuando w3 se apaga por completo, el efecto es más severo porque la capacidad cae de tres nodos a dos y además hay llamadas que fallan por conexión rechazada; aun así, el cien por ciento se atiende y el circuito de w3 queda OPEN con dieciséis fallos registrados. Al reiniciarlo, el estado HALF_OPEN permite una petición de prueba, esta tiene éxito y el nodo vuelve a CLOSED: en el escenario de recuperación w3 recibió sesenta pasos adicionales y el throughput regresó a 23,53 rps con un p95 de 519 ms.

El escenario de guardrail es el que valida la Saga. El veinticinco por ciento de las transacciones fue rechazado por la regla de negocio en la etapa de cumplimiento; las treinta y una transacciones afectadas se compensaron en orden inverso y ninguna quedó a medias. La tasa de atención fue del cien por ciento y los errores de sistema, cero. El p95 sube a 1 279 ms porque la compensación agrega dos llamadas de cincuenta milisegundos y porque el rechazo ocurre después de ejecutar dos pasos. Es un costo aceptable: la alternativa, dejar los pasos previos sin revertir, produciría inconsistencia.

Costo por paso y sobrecarga de coordinación

Figura 8

Costo promedio y p95 de cada paso de la Saga en la línea base



Nota. Elaboración propia. La línea base corresponde a tres nodos sin fallos y 120 transacciones.

En la línea base, los promedios de los tres pasos suman 573 ms (142,6 + 245,9 + 184,9), mientras que la latencia extremo a extremo promedio es de 831,8 ms. La diferencia de 258 ms, un 31 % del total, es la sobrecarga de coordinación: tres viajes HTTP adicionales, serialización y deserialización de JSON, publicación de eventos y, sobre todo, espera en la cola de despacho. Con veinticuatro clientes concurrentes y doce espacios de cómputo, cada transacción pasa una parte de su vida esperando turno. Esa espera no aparece en la métrica del paso, porque el paso solo se mide cuando el trabajador lo ejecuta, pero sí aparece en la latencia que percibe el cliente.

La separación entre latencia de servicio y latencia de cola es fundamental para interpretar el sistema. Si el equipo hubiera optimizado solo los pasos, habría mejorado una fracción del tiempo total y habría ignorado el verdadero cuello de botella. La evidencia sugiere que, con la

carga actual, el coordinador y la cola de despacho son el componente a optimizar antes que el cómputo. Duplicar los hilos de despacho o aislar un pool por trabajador son las dos alternativas que se discuten en los ajustes.

Ajustes derivados de la medición

El equipo aplicó cinco ajustes, cada uno motivado por una observación y verificado con una medición posterior. No se presentan como mejoras hipotéticas sino como cambios efectivamente incorporados al prototipo.

Ajuste 1: resolución de nombres en Windows

La primera ejecución de humo mostró una latencia extremo a extremo de 8 556 ms con tres nodos y una tasa de 0,70 transacciones por segundo, un resultado incompatible con la arquitectura diseñada. El diagnóstico descartó el código de la Saga y apuntó a la resolución de nombres: cada petición a `localhost` tardaba alrededor de dos segundos porque el sistema intentaba primero la dirección IPv6 y luego reintentaba con IPv4.

Tabla 15

Costo por petición HTTP según el nombre de host

Host	Promedio por petición	Mejora
`localhost`	2 041,84 ms	1,0×
`127.0.0.1`	2,15 ms	951,5×

El cambio de `localhost` por `127.0.0.1` en todos los nodos redujo el costo por petición de 2 042 ms a 2,15 ms, una mejora de 951,5 veces medida con el script `medir_resolucion.py`. Como cada transacción hace tres llamadas, el ahorro por transacción es de aproximadamente seis segundos. La lección arquitectónica es que la latencia de resolución de nombres es parte de la latencia de red y debe medirse, no suponerse; en un despliegue con DNS interno o service discovery el efecto puede reaparecer y exige caché o resolución directa.

Ajuste 2: estrategia de balanceo consciente de la carga

El escenario de latencia en w2 reveló que round-robin degrada el throughput un 65,5 %. Para verificar la causa se ejecutó un experimento comparativo con la misma carga y el mismo fallo, cambiando solo la estrategia.

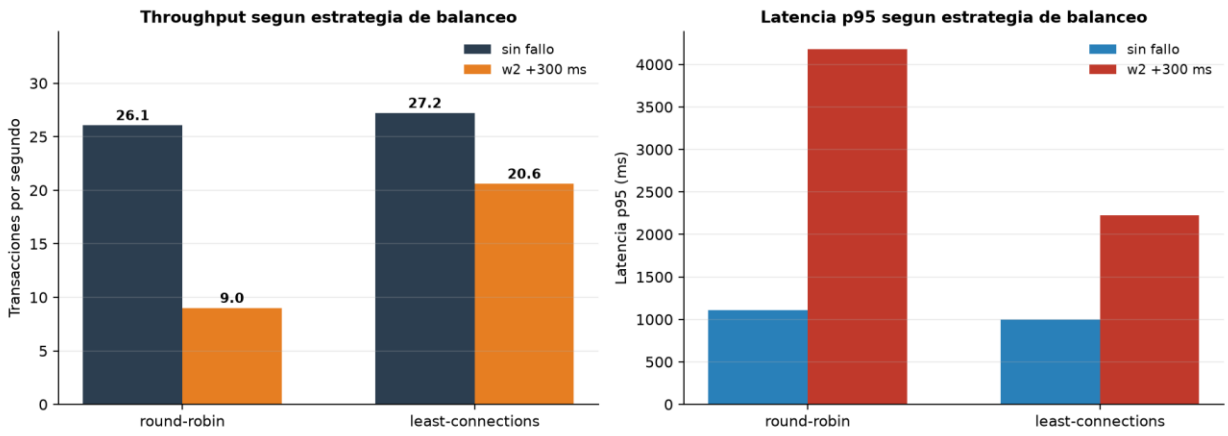
Tabla 16

Comparación de estrategias con y sin un nodo lento

Estrategia	Base (rps)	Base p95 (ms)	w2 +300 ms (rps)	w2 +300 ms p95 (ms)	Degradación
round-robin	26,05	1 111,9	8,99	4 177,7	65,5 %
least-connections	27,20	997,6	20,59	2 223,9	24,3 %

Figura 9

Throughput y latencia p95 según la estrategia de balanceo



Nota. Elaboración propia. Ambos experimentos usan 120 transacciones, concurrencia 24 y 300 ms de latencia inyectada en w2.

Con least-connections, el throughput bajo fallo asciende a 20,59 rps, un 129 % más que round-robin, y el p95 baja de 4 178 ms a 2 224 ms, una reducción del 46,8 %. El mecanismo es simple: el nodo lento acumula peticiones en vuelo y la estrategia deja de enviarle trabajo nuevo, mientras round-robin insiste en repartir por igual. La conclusión es que el patrón Strategy no es

un adorno de diseño: en un sistema con latencias heterogéneas, la política de balanceo es una decisión de resiliencia. Por esa razón el prototipo permite seleccionar la estrategia al arrancar y la suite principal conserva round-robin para documentar el peor caso.

Ajuste 3: broker con backpressure y consumidor de auditoría

La primera versión del broker publicaba eventos y escribía la bitácora, pero no tenía consumidor. Al final de la suite la cola había acumulado 9 737 eventos, la profundidad era igual al total publicado y el contador de consumidos era cero. El sistema funcionaba, pero estaba a 263 eventos de alcanzar el límite de diez mil y empezar a descartar.

Se aplicaron dos cambios. Primero, una cola acotada con política de descarte del evento más antiguo, que garantiza que el productor nunca se bloquee y que el descarte sea visible en la métrica `dropped`. Segundo, un suscriptor de auditoría que drena el topic, cuenta los eventos por tipo y registra la marca del último evento. Después del ajuste, la suite publicó y consumió 9 735 eventos, con cero descartes y profundidad de cola cero al final de cada escenario. La lección es que un broker sin consumidor es una fuga de memoria con forma de arquitectura; el patrón Broker solo se completa cuando existe al menos un suscriptor y una política explícita para la saturación (Hohpe & Woolf, 2003).

Ajuste 4: distinción entre error de negocio y error de nodo

En la primera versión, cualquier respuesta de error de un trabajador se trataba igual: se reintentaba en otro nodo y se contaba como fallo del nodo. Eso producía dos efectos indeseables. Un rechazo de guardrail se reintentaba dos veces, aunque el resultado fuera determinista, y el Circuit Breaker acumulaba fallos por una regla de negocio, lo que podía abrir el circuito de un nodo sano.

El ajuste introdujo el campo `kind` en la respuesta del trabajador y una rama explícita en el despachador: si el error es de negocio, la Saga se compensa de inmediato y el nodo no recibe penalización; si el error es de infraestructura, se reintenta en otro nodo y se registra el fallo. La evidencia está en el escenario de guardrail: treinta y una compensaciones, cero errores de sistema y el circuito de w2 en CLOSED, es decir, sin falsos positivos. La lección es que la taxonomía de errores es una decisión de arquitectura: reintentar lo que no debe reintentarse es tan dañino como no reintentar lo que sí debe reintentarse.

Ajuste 5: dimensionamiento de los pools

La sobrecarga de coordinación del 31 % mostró que la cola de despacho es el recurso escaso. El equipo mantuvo dieciséis hilos de despacho para doce espacios de cómputo porque las llamadas son de entrada y salida y conviene tener margen para absorber picos; sin embargo, la evidencia indica que el dimensionamiento óptimo depende de la heterogeneidad de la latencia. Con nodos homogéneos, un pool cercano al número de espacios de cómputo reduce la contención; con nodos heterogéneos, un pool mayor absorbe la espera pero arriesga saturarse. La recomendación para una siguiente iteración es aislar un pool por trabajador, de modo que un nodo lento no consuma los hilos destinados a los nodos sanos. Esa sería la versión completa del patrón Bulkhead en el coordinador, que hoy solo está aplicado en los trabajadores.

Cumplimiento de las metas

Tabla 17

Contraste entre las metas definidas y los resultados medidos

Código	Meta	Resultado	Estado
RNF-01	Throughput ≥ 2,5× con tres nodos	2,89×	Cumplida
RNF-02	100 % atendido con un nodo caído	100 %	Cumplida

Código	Meta	Resultado	Estado
RNF-03	p95 < 1 500 ms con tres nodos	1 043,4 ms	Cumplida
RNF-04	Recuperación del nodo en menos de 10 s	Menos de 6 s	Cumplida
RNF-05	Toda Saga completada o compensada	1 200 transacciones, 0 errores	Cumplida
RNF-06	Un evento por transición, sin pérdidas	9 735 eventos, 0 descartes	Cumplida
RNF-07	Ejecutable con biblioteca estándar	Cero dependencias en ejecución	Cumplida
RNF-08	Un módulo por responsabilidad	Nueve módulos especializados	Cumplida

Las ocho metas se cumplieron. Dos de ellas, la de escalabilidad y la de latencia, se superaron con margen; la de recuperación se cumplió con holgura porque el enfriamiento del Circuit Breaker es de cinco segundos y el escenario esperó seis antes de medir. La meta de trazabilidad se cumplió con una salvedad: el broker es un punto único de fallo y la bitácora es local, de modo que la garantía es de proceso vivo y no de durabilidad ante caída del broker. Esa limitación se declara en el apartado siguiente.

Limitaciones y validez

Los resultados deben interpretarse dentro de sus condiciones de validez. La primera limitación es el entorno: todos los procesos se ejecutaron en un solo equipo con cuatro núcleos lógicos, de modo que la contención de CPU y la competencia por el disco afectan las mediciones. En un despliegue con nodos en máquinas separadas, el throughput absoluto sería distinto, aunque las tendencias —escalado casi lineal, efecto del breaker, costo de la compensación— deberían mantenerse.

La segunda limitación es la naturaleza sintética de las etapas cognitivas. Las latencias simuladas permiten comparar escenarios, pero no representan la distribución real de un modelo de lenguaje, que suele tener colas pesadas y errores intermitentes. Un modelo real introduciría

además límites de cuota y costos variables. La tercera limitación es la ausencia de latencia de red: en local, una llamada entre nodos cuesta alrededor de dos milisegundos; en una red real, la latencia y la pérdida de paquetes cambiarían el equilibrio entre coordinación y cómputo.

La cuarta limitación es la durabilidad. El estado de las Sagas vive en memoria del coordinador y la bitácora en un archivo local; si el coordinador falla, las Sagas en vuelo se pierden, y si el broker falla, se pierde la trazabilidad no consumida. La quinta es la seguridad: el prototipo no implementa autenticación, autorización ni cifrado, decisiones aceptables en un ejercicio académico pero inadmisibles en producción. Finalmente, la variabilidad de las mediciones es real: corridas sucesivas del mismo escenario difieren entre un dos y un cinco por ciento, por lo que las comparaciones se hacen sobre diferencias mayores y no sobre decimales aislados.

Conclusiones

La Actividad 4 se propuso llevar NexusAI del plano conceptual al plano operativo, y ese tránsito se cumplió: la arquitectura que en la Actividad 2 era un conjunto de diagramas y promesas es hoy un clúster de seis procesos que se levanta con un comando, atiende transacciones reales por HTTP, publica eventos en un broker, se degrada con criterio cuando un nodo falla y se recupera sin intervención manual. El aporte central del ejercicio no es el prototipo en sí, sino la cadena de evidencia que une cada decisión de diseño con una medición reproducible. Esa cadena es la que permite afirmar que la arquitectura funciona, y no solo que está bien dibujada.

El objetivo general se cumplió con holgura medible. El escalado horizontal pasó de 9,24 a 26,74 transacciones por segundo al triplicar los nodos, un speedup de 2,89× con una eficiencia del 96,5 %, lo que confirma que la separación entre gateway, broker y trabajadores permite

crecer en capacidad sin rediseñar. La latencia p95 cayó de 3 185 ms a 1 043 ms en el mismo experimento, de modo que el crecimiento no se pagó con lentitud. La robustez fue igualmente consistente: en los seis escenarios de fallo se atendió el cien por ciento de las 1 200 transacciones de la suite y no se registró un solo error de sistema. Un nodo que falla el cien por ciento de sus peticiones, un nodo apagado y un veinticinco por ciento de rechazos de negocio produjeron degradación de capacidad, nunca pérdida de servicio. Esa es, en términos de Bass et al. (2022), la diferencia entre una arquitectura que promete disponibilidad y una que la demuestra bajo estímulo.

Los patrones seleccionados no fueron ornamentales; cada uno resolvió un problema identificado y su efecto se puede señalar en los datos. La Saga orquestada permitió que treinta y una transacciones rechazadas por el guardrail se compensaran en orden inverso sin dejar estado inconsistente, que es exactamente el problema que Garcia-Molina y Salem (1987) describieron cuando no existe una transacción distribuida global. El Circuit Breaker y el Retry convirtieron un nodo defectuoso en una pérdida temporal de capacidad en lugar de un fallo en cascada, tal como plantea Nygard (2018). El Broker desacopló la producción de eventos de su consumo y, con el consumidor de auditoría, cerró el ciclo que Hohpe y Woolf (2003) consideran indispensable para que el patrón esté completo. El Proxy o API Gateway dio una interfaz única y un lugar donde aplicar políticas transversales, en la línea de Fowler (2014) y Richardson (2018). El patrón Strategy, finalmente, demostró ser una decisión de resiliencia y no un detalle cosmético: con un nodo lento, least-connections sostuvo 20,59 rps frente a 8,99 de round-robin, una diferencia del 129 %.

El ejercicio también produjo aprendizajes que no estaban en el plan y que probablemente sean lo más valioso del trabajo. El primero es que la latencia de resolución de nombres es

infraestructura, no configuración: cambiar `localhost` por `127.0.0.1` redujo el costo por petición de 2 042 ms a 2,15 ms, un factor de 951,5, y sin ese hallazgo el prototipo habría parecido una arquitectura defectuosa cuando el problema estaba en el sistema operativo. El segundo es que un broker sin consumidor es una fuga de memoria disfrazada de arquitectura: la cola crecía hasta rozar el límite de diez mil elementos y el sistema seguía respondiendo, lo que ocultaba el riesgo. El tercero, y el más incómodo, es que el Bulkhead aplicado solo en los trabajadores no protege al coordinador: con latencia inyectada, el recurso que se agota es el pool de despacho compartido, no la capacidad de cómputo. Ese hallazgo obligó a distinguir entre proteger la ejecución y proteger la espera, una lección que no aparece en los manuales con la claridad con que aparece en una gráfica de latencia.

El trabajo en equipo se organizó por responsabilidades verificables y no por reparto informal de tareas. La construcción del informe se apoyó en un generador propio que toma el texto fuente y produce el documento con formato APA 7, lo que permitió que ambos integrantes editaran contenido sin pelear con estilos, y los diagramas se versionaron como texto PlantUML para que cualquier cambio fuera revisable y reproducible (PlantUML, 2024). Esa disciplina de artefactos generados tiene un efecto secundario valioso: el informe, los diagramas y las gráficas se regeneran con un comando, de modo que la evidencia siempre corresponde a la última versión del código. La reproducibilidad dejó de ser una intención y se volvió una propiedad del repositorio.

Quedan limitaciones que conviene declarar sin adornos. El prototipo se ejecutó en un solo equipo con cuatro núcleos lógicos, de modo que los valores absolutos de throughput están acotados por la contención de CPU y no por la arquitectura; las tendencias son válidas, los números no son universales. Las etapas cognitivas son simuladas, de modo que el sistema no

enfrenta la distribución de colas pesadas ni los errores intermitentes de un modelo de lenguaje real. El estado de las Sagas vive en memoria y la bitácora en un archivo local, lo que significa que la durabilidad es de proceso vivo y no de caída. Y la seguridad está ausente por completo: no hay autenticación, autorización ni cifrado, decisiones defendibles en un ejercicio académico e inadmisibles en cualquier entorno real.

El trabajo futuro se ordena por impacto y no por facilidad. La primera prioridad es replicar el coordinador o, al menos, aislar un pool de despacho por trabajador, porque la evidencia señala ese componente como el límite del escalado. La segunda es persistir el estado de las Sagas y la bitácora en un almacenamiento durable, con un log de eventos que permita reconstruir el estado tras una caída, en la línea de Kleppmann (2017). La tercera es sustituir las latencias simuladas por servicios cognitivos reales, con sus cuotas, sus colas y sus fallos, para validar si las conclusiones sobreviven al contacto con la realidad. La cuarta es incorporar seguridad de extremo a extremo y trazas distribuidas con identificadores de correlación, sin las cuales un sistema de este tipo no es operable en producción (Newman, 2021). Ninguna de estas tareas cambia la arquitectura; todas la completan.

La conclusión final es que tejer redes entre hilos y nodos no consiste en añadir concurrencia ni en multiplicar procesos, sino en decidir dónde se paga la complejidad y cómo se demuestra que esa decisión funciona. NexusAI 2.0 paga la complejidad en un coordinador centralizado y la compensa con visibilidad total del estado de cada transacción. Esa elección tiene costos —un punto único de despacho, estado en memoria, dependencia del broker— y el informe los declara junto con las mediciones que los cuantifican. Una arquitectura honesta no es la que no tiene debilidades, sino la que las conoce, las mide y las deja escritas para quien venga

después. Eso es lo que este ejercicio intentó hacer, y en esa medida considera cumplidos sus objetivos.

Referencias

Bass, L., Clements, P., & Kazman, R. (2022). *Software architecture in practice* (4th ed.). Addison-Wesley.

Fowler, M. (2014, 25 de marzo). *Microservices*. https://martinfowler.com/articles/microservices.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Garcia-Molina, H., & Salem, K. (1987). Sagas. *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 249–259. https://doi.org/10.1145/38713.38742

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

International Organization for Standardization. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. https://www.iso.org/standard/35733.html

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

PlantUML. (2024). *PlantUML language reference guide*. https://plantuml.com/guide

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Anexo A. Contratos HTTP del prototipo

El contrato es uniforme en los cuatro tipos de nodo: cuerpo JSON, respuesta JSON y códigos de estado que distinguen el resultado de negocio del de infraestructura. Las tablas siguientes resumen los puntos de acceso y los campos de cada mensaje.

Tabla 18

Puntos de acceso HTTP expuestos por cada tipo de nodo

Nodo	Método y ruta	Función	Códigos de respuesta
Coordinador	`POST /task`	Inicia una Saga completa	200, 409, 503
Coordinador	`GET /metrics`	Métricas agregadas del clúster	200
Coordinador	`POST /reset`	Reinicia métricas y breakers	200
Coordinador	`GET /dashboard.html`	Tablero de observabilidad	200
Trabajador	`POST /execute`	Ejecuta un paso del pipeline	200, 500
Trabajador	`GET /metrics`	Métricas locales y estado del nodo	200
Trabajador	`POST /chaos`	Inyecta latencia, fallos o caída	200
Broker	`POST /publish`	Publica un mensaje en un topic	200, 503
Broker	`GET /consume`	Extrae un mensaje del topic	200, 204
Broker	`GET /metrics`	Profundidad de cola y contadores	200

La petición de una transacción acepta un cuerpo libre en `payload` y un canal de origen. El coordinador no impone un esquema rígido al contenido porque el prototipo no ejecuta un modelo real; lo que sí garantiza es que toda petición produzca una respuesta con trazabilidad completa o un rechazo explícito.

```
POST /task HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json

{
  "payload": {
    "prompt": "escalar incidente critico #42",
    "canal": "whatsapp"
```

```
  }
}
```

La respuesta exitosa devuelve el identificador de la Saga, la lista de pasos con el nodo que los ejecutó y la latencia de cada uno, además de la duración interna y la latencia extremo a extremo medida por el coordinador.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "completed",
  "saga_id": "e77feb53a1",
  "steps": [
    {"step": "triage",     "worker": "w1", "attempt": 1, "latency_ms": 118.4},
    {"step": "rag",        "worker": "w2", "attempt": 1, "latency_ms": 231.7},
    {"step": "compliance", "worker": "w2", "attempt": 1, "latency ms": 134.9}
  ],
  "duration ms": 501.8,
  "e2e_latency_ms": 508.2
}
```

Cuando la regla de negocio rechaza la transacción, el código es 409 y el cuerpo incluye el paso fallido, el motivo y el número de pasos compensados. Cuando no hay ningún nodo disponible, el código es 503 y el cuerpo indica que todos los circuitos están abiertos. La diferencia entre ambos casos es la que permite al cliente decidir si corrige la entrada o reintenta más tarde.

```
HTTP/1.1 409 Conflict

{
  "status": "rolled_back",
  "saga_id": "7bdadaa043",
  "failed_step": "compliance",
  "reason": "guardrail: confianza semantica < 0.90",
  "compensated_steps": 2,
  "duration_ms": 673.9
}
```

Anexo B. Extracto de la bitácora de eventos

La bitácora es un archivo JSONL en el que cada línea es un sobre con marca de tiempo, topic y mensaje. Se incluyen tres líneas representativas: el inicio de una Saga, una compensación por rechazo de guardrail y el cierre exitoso de otra transacción.


```
{"ts": 1794267910808.94, "topic": "events", "message": {"event": "saga.started",
 "source": "coordinator", "ts": 1794267910757.16, "saga_id": "ddefa5c333", "canal":
"web"}}
{"ts": 1794267974934.96, "topic": "events", "message": {"event": "saga.rolled back",
 "source": "coordinator", "ts": 1794267974848.66, "saga_id": "7bdadaa043",
 "failed step": "compliance", "reason": "guardrail: confianza semantica < 0.90",
 "compensated_steps": 2, "duration_ms": 673.93}}
{"ts": 1794267998822.08, "topic": "events", "message": {"event": "saga.completed",
 "source": "coordinator", "ts": 1794267998812.62, "saga id": "9b8b797e4e",
 "duration_ms": 482.17}}
```


El consumidor de auditoría contó, sobre el total de la suite, 1 212 eventos `saga.started`, 3 605 `saga.step.completed`, 3 667 `task.completed`, 1 180 `saga.completed`, 40 `task.failed` y 31 `saga.rolled_back`. La suma coincide con los 9 735 eventos publicados y consumidos, sin descartes. La relación entre eventos de inicio y de cierre no es exactamente uno a uno porque los contadores del broker son acumulativos y algunos eventos quedaron en vuelo al terminar el último escenario.

Tabla 19

Eventos registrados por tipo durante la suite completa

Tipo de evento	Cantidad	Interpretación
`saga.started`	1 212	Sagas iniciadas
`saga.step.completed`	3 605	Pasos ejecutados con éxito
`task.completed`	3 667	Confirmaciones de los trabajadores
`saga.completed`	1 180	Sagas cerradas con éxito
`task.failed`	40	Fallos de infraestructura inyectados
`saga.rolled_back`	31	Sagas compensadas por guardrail

Anexo C. Salida de la suite de evaluación

La suite imprime un bloque por escenario con tareas, completadas, compensadas, errores, throughput, tiempo de pared y percentiles. Se reproduce la salida de la corrida que alimenta las tablas de escalabilidad, de robustez y de comparación de estrategias del informe.

```
=== escalabilidad_1_nodo (1 nodo(s) de computo con la misma carga) ===
tareas= 180  ok= 180  compensadas=   0  errores=  0
throughput=  9.24 rps   wall= 19.47s   handled=100.00%
latencia e2e: avg=2469.2ms  p50=2131.3ms  p95=3185.0ms  p99=3243.4ms  max=3251.2ms

=== escalabilidad 2 nodos (2 nodo(s) de computo con la misma carga) ===
tareas= 180  ok= 180  compensadas=   0  errores=  0
throughput= 18.14 rps   wall=  9.92s   handled=100.00%
latencia e2e: avg=1244.3ms  p50=1047.7ms  p95=1617.0ms  p99=1638.8ms  max=1671.3ms

=== escalabilidad 3 nodos (3 nodo(s) de computo con la misma carga) ===
tareas= 180  ok= 180  compensadas=   0  errores=  0
throughput= 26.74 rps   wall=  6.73s   handled=100.00%
latencia e2e: avg=844.7ms  p50=854.2ms  p95=1043.4ms  p99=1127.0ms  max=1157.9ms

=== robustez linea base (3 nodos sin fallos inyectados) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput= 26.37 rps   wall=  4.55s   handled=100.00%
latencia e2e: avg=831.8ms  p50=843.5ms  p95=1033.6ms  p99=1099.5ms  max=1207.0ms

=== robustez_latencia_w2 (300 ms de latencia inyectada en el nodo w2) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput=  8.83 rps   wall= 13.59s   handled=100.00%
latencia e2e: avg=2419.7ms  p50=2546.7ms  p95=4128.4ms  p99=4945.4ms  max=5320.0ms

=== robustez_nodo_defectuoso_w2 (w2 falla el 100% de sus peticiones) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput= 17.87 rps   wall=  6.71s   handled=100.00%
latencia e2e: avg=1226.3ms  p50=1258.3ms  p95=1531.2ms  p99=1624.0ms  max=1649.1ms

=== robustez_guardrail (25% de transacciones rechazadas por guardrail) ===
tareas= 120  ok=  89  compensadas=  31  errores=  0
throughput= 23.82 rps   wall=  5.04s   handled=100.00%
latencia e2e: avg=914.6ms  p50=913.9ms  p95=1279.2ms  p99=1377.5ms  max=1498.7ms

=== robustez_caida_w3 (w3 apagado por completo durante la carga) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput= 14.55 rps   wall=  8.25s   handled=100.00%
latencia e2e: avg=1533.8ms  p50=1323.5ms  p95=2828.8ms  p99=3453.5ms  max=3648.3ms

=== robustez recuperacion w3 (w3 reiniciado; el Circuit Breaker vuelve a enrutar
trafico) ===
tareas=  60  ok=  60  compensadas=   0  errores=  0
throughput= 23.53 rps   wall=  2.55s   handled=100.00%
latencia e2e: avg=475.2ms  p50=470.6ms  p95=518.6ms  p99=570.4ms  max=595.0ms
```

Los artefactos de la corrida son `resultados.json` con las métricas completas por escenario, `resumen.md` con la tabla consolidada, `event_log.jsonl` con la bitácora y cinco gráficas en `pruebas/graficas/`. El experimento comparativo de estrategias se ejecuta aparte

con `experimento_estrategias.py` y produce `estrategias.json` y la gráfica `05_estrategias.png`.

Anexo D. Reproducción del entorno

El sistema se ejecuta con Python 3.14 y su biblioteca estándar; las dependencias externas solo son necesarias para generar las gráficas y los diagramas. La tabla siguiente resume los comandos y el resultado esperado de cada uno.

Tabla 20

Comandos de reproducción y resultado esperado

Paso	Comando	Resultado esperado
1	`python run_cluster.py --workers 3`	Seis procesos en línea y tablero en el puerto 8088
2	Abrir `http://127.0.0.1:8088/dashboard.html`	Tablero con métricas en vivo
3	`python load_test.py --tasks 120 --concurrency 24`	Bloque con throughput y percentiles
4	`python chaos_test.py latency --worker http://127.0.0.1:8102 --ms 300`	Nodo w2 degradado
5	`python chaos_test.py fail --worker http://127.0.0.1:8102 --rate 1.0`	Breaker de w2 en OPEN
6	`python chaos_test.py reset`	Nodos restablecidos
7	`python benchmark.py`	Suite completa y artefactos en `pruebas/`
8	`python ../pruebas/experimento_estrategias.py`	Comparación de estrategias y gráfica 05
9	`python ../diagramas/render_diagrams.py`	Diagramas PNG y SVG regenerados
10	`python build_reporte.py`	Informe `.docx` regenerado

La versión de Python utilizada fue 3.14.7 sobre Windows 11 con cuatro núcleos lógicos. Las gráficas requieren `matplotlib` y `numpy`; los diagramas se renderizaron con el servicio público de PlantUML y el script `render_diagrams.py` incluido en el repositorio. El informe se genera con `python-docx` y `Pillow`. Ninguna de estas dependencias interviene en la ejecución del clúster, de modo que el prototipo se puede demostrar en un equipo limpio con solo el intérprete de Python.

Anexo E. Enlaces de entrega

El repositorio y el video se entregan como complementos del informe. El repositorio contiene el código fuente, los diagramas en formato PlantUML, las pruebas, las gráficas y los scripts de generación; el video muestra la ejecución del clúster, la inyección de fallos y la lectura del tablero.

Tabla 21

Enlaces de los entregables complementarios

Entregable	Ubicación
Repositorio del código	Pendiente de publicar
Video demostrativo (YouTube, no listado)	Pendiente de publicar
Informe en PDF	Documento adjunto en el aula virtual

El video se configuró como no listado para que solo las personas con el enlace puedan acceder, conforme a la indicación de la docente. La demostración sigue el mismo orden del Anexo D: arranque del clúster, carga base, latencia inyectada, fallo total de un nodo, compensación por guardrail, recuperación y lectura del tablero. Cada escenario se muestra con la salida de la consola y el tablero en paralelo, de modo que la evidencia visual coincida con las cifras del informe.